ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

KIẾN TRÚC PHẦN MỀM (CO3017)

BÀI TẬP LỚN

INTELLIGENT RESTAURANT MANAGEMENT SYSTEM

Giảng viên hướng dẫn: Trần Trương Tuấn Phát
Lớp: L01
Sinh viên thực hiện: Lữ Quốc Pháp - 2312562
Nguyễn Lê Nguyên - 2312365
Trần Trọng Nghĩa - 2312285
Lê Sĩ Nhật Quang - 2312791
Nguyễn Đức Phương - 2312749

THÀNH PHỐ HỒ CHÍ MINH, 2026

Mục lục

Danh mục hình vẽ **3**

Danh mục bảng biểu **3**

Danh sách thành viên và công việc **4**

1 Bối cảnh hệ thống Intelligent Restaurant Management System **5**

1.1 Giới thiệu và mục tiêu 5

1.2 Phạm vi hệ thống và các phân hệ chức năng 5

1.2.1 Đặt món số và quản lý thực đơn 5

1.2.2 Hiển thị đơn bếp và điều phối luồng chế biến 6

1.2.3 Quản lý bàn và đặt chỗ 6

1.2.4 Tính tiền, thanh toán và xuất hóa đơn 6

1.2.5 Giám sát tồn kho 7

1.2.6 Phân tích và báo cáo 7

1.2.7 Công cụ quản trị 7

1.3 Tác nhân và tương tác vận hành điển hình 7

1.4 Yêu cầu chức năng 8

1.5 Yêu cầu phi chức năng 9

1.6 Ràng buộc và giả định 10

1.7 Tiêu chí thành công 10

2 Use Case Scenarios - System Modeling **11**

3 So sánh và lựa chọn phong cách kiến trúc phù hợp (Compare and choose suitable architecture styles) **32**

3.1 Modular Monolith. 32

3.2 Microservices. 32

3.3 Event-driven. 33

3.4 Kết luận lựa chọn. 33

4 Xây dựng kiến trúc phần mềm cho hệ thống 34

4.1 Module Views 34

4.1.1 Context View 34

4.1.2 Package View 36

4.1.3 Module View 38

4.2 Component & Connector Views 39

4.2.1 Component View 39

4.2.2 Component & Connector (C&C) View 43

4.3 Allocation View 46

4.3.1 Install Style 47

4.3.2 Deployment Style 49

4.3.3 Work Assignment Style 50

5 Thiết kế class diagram 52

5.1 Nghiệp vụ 1: Nhân viên thực hiện chọn món và tạo order 52

5.2 Nghiệp vụ 2: Quản trị viên thiết lập Catalog (Menu và Khuyến mãi) . . 54

5.3 Nghiệp vụ 3: Xử lý nhà bếp và Tự động trừ tồn kho 56

5.4 Nghiệp vụ 4: Thanh toán hóa đơn và Quản lý trạng thái bàn 58

5.5 Nghiệp vụ 5: Phân tích dữ liệu và Xuất báo cáo 60

5.6 Nghiệp vụ 6: Kiểm soát truy cập và Lưu vết hệ thống (Cross-Cutting) . 62

Danh mục hình vẽ

1 Context View của hệ thống 34
2 Package View của hệ thống 36
3 Module View của hệ thống 38
4 Component View của hệ thống 39
5 Component & Connector View của hệ thống 43
6 Install Style của hệ thống 47
7 Deployment Style của hệ thống 49
8 Thiết kế class diagram cho nghiệp vụ tạo order 52
9 Thiết kế class diagram cho nghiệp vụ quản lý danh mục và khuyến mãi . 54
10 Thiết kế class diagram cho nghiệp vụ chế biến và quản lý nguyên liệu . . 56
11 Thiết kế class diagram cho nghiệp vụ thanh toán và giải phóng bàn . . . 58
12 Thiết kế class diagram cho nghiệp vụ phân tích và tổng hợp báo cáo . . . 60
13 Thiết kế class diagram cho hạ tầng phân quyền và Audit 62

Danh mục bảng biểu

1 *Danh sách thành viên và công việc* 4

Danh sách thành viên và công việc

STT	Họ và tên	MSSV	Tổng số công việc	Hoàn thành
1	Lữ Quốc Pháp	2312562	<Blank>	100%
2	Nguyễn Lê Nguyên	2312365	<Blank>	100%
3	Lê Sĩ Nhật Quang	2312791	<Blank>	100%
4	Trần Trọng Nghĩa	2312285	<Blank>	100%
5	Nguyễn Đức Phương	2312749	<Blank>	100%

Bảng 1: ***Danh sách thành viên và công việc***

1 Bối cảnh hệ thống Intelligent Restaurant Management System

1.1 Giới thiệu và mục tiêu

Hệ thống IRMS (Intelligent Restaurant Management System) được đặc tả như một ứng dụng phần mềm nhằm tối ưu hóa và tự động hóa các hoạt động vận hành cốt lõi trong môi trường nhà hàng. Thông qua các giao diện số, cơ sở dữ liệu dùng chung và cơ chế điều phối dựa trên logic phần mềm, các quy trình như duyệt thực đơn, tiếp nhận và xử lý đơn hàng, quản lý trạng thái bàn, cập nhật tồn kho, cũng như tổng hợp dữ liệu vận hành được hỗ trợ một cách nhất quán. Trên cơ sở đó, sự tương tác giữa khối phục vụ (front-of-house), khối bếp (back-of-house) và bộ phận quản lý được đồng bộ hóa nhằm giảm thiểu sai lệch thông tin, hạn chế độ trễ trong luồng xử lý đơn hàng và cải thiện tính dự đoán của thời gian phục vụ.

Mục tiêu kỹ thuật của hệ thống được đặt ra bao gồm: (i) chuẩn hóa quy trình tiếp nhận đơn hàng và truyền đạt yêu cầu chế biến, (ii) tăng cường khả năng phối hợp giữa các trạm chế biến thông qua cơ chế định tuyến và ưu tiên công việc, (iii) nâng cao năng lực giám sát trạng thái phục vụ tại bàn và trạng thái thanh toán, và (iv) cung cấp các công cụ phân tích phục vụ ra quyết định quản trị dựa trên dữ liệu. Hệ thống được định hướng để thích ứng với các điều kiện phục vụ biến động (giờ cao điểm, đơn hàng số lượng lớn) bằng cách hỗ trợ điều chỉnh luồng công việc nhằm hạn chế tắc nghẽn và giảm thiểu lỗi tác nghiệp.

1.2 Phạm vi hệ thống và các phân hệ chức năng

Phạm vi của IRMS được giới hạn trong các chức năng vận hành chính của nhà hàng, trong đó các phân hệ sau được xem là cấu phần trọng yếu.

1.2.1 Đặt món số và quản lý thực đơn

Quá trình đặt món được hỗ trợ thông qua thiết bị đầu cuối (tablet/terminal), nhờ đó việc ghi nhận đơn hàng được thực hiện nhanh chóng và có cấu trúc. Thực đơn được

cho phép cập nhật theo thời gian thực, bao gồm thay đổi giá, cấu hình khuyến mãi và đánh dấu món không khả dụng. Đơn hàng được biểu diễn với khả năng mang theo các ràng buộc và tham số tùy biến, bao gồm ghi chú dị ứng, yêu cầu đặc biệt, lựa chọn theo combo, cũng như các cấu hình biến thể (kích cỡ, topping, mức chín). Nhằm mục đích đảm bảo tính toàn vẹn thông tin đầu vào, các tùy chọn này được xem như một phần dữ liệu bắt buộc phải được truyền đến khối bếp trong toàn bộ vòng đời xử lý đơn hàng.

1.2.2 Hiển thị đơn bếp và điều phối luồng chế biến

Đơn hàng sau khi được ghi nhận được định tuyến tự động đến hệ thống hiển thị bếp (KDS – Kitchen Display System). Tại đây, các đơn được tổ chức theo tiêu chí thời gian chuẩn bị, nhóm món và trạm chế biến (ví dụ: grill, fryer, dessert) nhằm giảm thiểu xung đột tài nguyên và hỗ trợ phân công tác vụ. Hơn nữa, các cơ chế cảnh báo trực quan được sử dụng để thông báo sự kiện đơn hàng mới và nhận diện nguy cơ trễ hạn theo ngưỡng thời gian mục tiêu. Trạng thái xử lý món được cho phép cập nhật theo các mốc tiến độ (`Started` → `Cooking` → `Ready to Serve`), qua đó tạo tiền đề cho việc đồng bộ hóa thông tin phục vụ và giảm độ trễ trong việc giao món.

1.2.3 Quản lý bàn và đặt chỗ

Việc quản lý bàn được hỗ trợ trên sơ đồ số, trong đó trạng thái bàn (trống/đang sử dụng/đang dọn/đã đặt trước) được biểu diễn nhằm tối ưu hóa quy trình sắp xếp chỗ ngồi. Thời gian chờ dự kiến có thể được ước tính dựa trên dữ liệu lịch sử kết hợp trạng thái hiện tại của bàn và bếp. Mô-đun đặt chỗ được thiết kế để tiếp nhận đặt bàn theo lịch, xử lý danh sách chờ (waitlist) trong trường hợp vượt công suất, và hỗ trợ gửi thông báo cho khách theo kênh phù hợp với triển khai.

1.2.4 Tính tiền, thanh toán và xuất hóa đơn

Phân hệ thanh toán được thiết kế để hợp nhất các thành phần chi phí bao gồm món ăn, đồ uống, phí dịch vụ, khuyến mãi và thuế (tùy theo quy định áp dụng). Nhiều phương thức thanh toán được hỗ trợ, bao gồm tiền mặt, thẻ và ví điện tử. Ngoài ra, các cơ chế nghiệp vụ như tách hóa đơn theo khách/món và quản lý tiền tip được cung

cấp nhằm phù hợp với bối cảnh vận hành thực tế. Đối với các thao tác nhạy cảm như hoàn/huỷ thanh toán, kiểm soát theo vai trò và ghi nhận nhật ký (audit log) được xem là yêu cầu bắt buộc nhằm đảm bảo khả năng truy vết.

1.2.5 Giám sát tồn kho

Theo phạm vi được nêu, mức tiêu hao nguyên liệu được cập nhật thủ công sau các giai đoạn sơ chế hoặc phục vụ. Trên cơ sở đó, cảnh báo được kích hoạt khi tồn kho giảm xuống dưới ngưỡng định trước nhằm hỗ trợ ra quyết định bổ sung. Dữ liệu tiêu hao lịch sử được duy trì để phục vụ phân tích xu hướng sử dụng và hỗ trợ ước lượng nhu cầu tái đặt hàng, qua đó góp phần hạn chế thiếu hụt và lãng phí.

1.2.6 Phân tích và báo cáo

Các báo cáo bán hàng được tạo lập nhằm phản ánh giờ cao điểm, món bán chạy và hiệu năng doanh thu theo các khoảng thời gian (ngày/tuần/tháng). Ngược lại, các chỉ số vận hành được tổng hợp để nhận diện chậm trễ, vị trí tắc nghẽn tại bếp và hiệu suất của nhân sự theo vai trò. Dữ liệu báo cáo được hỗ trợ xuất ra nhằm phục vụ các mục tiêu kế toán, hoạch định và đánh giá hiệu năng.

1.2.7 Công cụ quản trị

Các cơ chế quản trị được triển khai thông qua kiểm soát truy cập theo vai trò (RBAC – Role-Based Access Control), qua đó quyền hạn được phân tách cho quản lý, phục vụ, bếp và thu ngân. Việc cấu hình thực đơn, cập nhật giá và chiến dịch khuyến mãi được tập trung hóa để giảm sai khác cấu hình giữa các điểm vận hành. Hệ thống nhật ký kiểm toán được duy trì để ghi nhận các hành vi quan trọng (ví dụ: hoàn tiền, huỷ đơn), nhằm cung cấp khả năng kiểm tra hậu nghiệm và hỗ trợ tuân thủ quy trình.

1.3 Tác nhân và tương tác vận hành điển hình

Các tác nhân chính được xác định bao gồm khách hàng, host, nhân viên phục vụ, nhân sự bếp, thu ngân và quản lý. Một luồng vận hành điển hình được mô tả như sau:

(i) trạng thái bàn và phiên phục vụ được khởi tạo khi khách được sắp xếp chỗ, (ii) đơn hàng được ghi nhận và định tuyến đến KDS để tổ chức chế biến theo trạm, (iii) trạng thái món được cập nhật liên tục để hỗ trợ phục vụ, (iv) hóa đơn được chốt và thanh toán được xử lý, và (v) dữ liệu tồn kho và dữ liệu phân tích được cập nhật nhằm phục vụ giám sát vận hành.

1.4 Yêu cầu chức năng

Các yêu cầu chức năng cốt lõi của IRMS được tổng hợp như sau:

1. **Xem menu (view):** hiển thị thực đơn trên menu màn hình của nhân viên
2. **Thực hiện order (create):** cho phép nhân viên ghi nhận order từ khách hàng và đưa order lên hệ thống
3. **Note đặc biệt trên order (customize):** cho phép nhân viên note: yêu cầu đặc biệt, dị ứng lựa chọn combo và tùy chỉnh theo ý khách hàng
4. **Quản lý thực đơn:** cho phép tạo/cập nhật/xóa (CRUD) món, cập nhật giá và đánh dấu món không khả dụng theo thời gian thực
5. **Cập nhật giá:** hệ thống cho phép quản trị viên thay đổi giá các món ăn
6. **Thực hiện khuyến mãi trên menu:** hệ thống cho phép quản trị viên thiết lập các chương trình khuyến mãi dựa trên các món trong menu hoặc các combo đặc biệt với giá ưu đãi
7. **Truyền order đến nhà bếp:** hệ thống phải cập nhật các order được tạo mới liên tục cho nhân viên nhà bếp thấy được
8. **Hiển thị danh sách order (View):** hệ thống phải hiển thị bảng thức ăn theo hàng đợi, theo loại thức ăn và theo prep station
9. **Quản lý order trong nhà bếp:** hệ thống phải thông báo mỗi khi có order mới và cảnh báo những order sắp hết deadline

10. **Cập nhật trạng thái order (Update):** hệ thống phải cho phép đầu bếp thay đổi trạng thái order theo các bước làm món để kiểm soát (Bắt đầu làm, đang nấu, Đã xong)

11. **Quản lý bàn và đặt chỗ:** theo dõi trạng thái bàn; ước tính thời gian chờ; quản lý reservation và waitlist

12. **Thanh toán và hóa đơn:** tổng hợp chi phí; hỗ trợ đa phương thức thanh toán; hỗ trợ tách hóa đơn và quản lý tip; xuất hóa đơn

13. **Tồn kho:** cập nhật tiêu hao thủ công; cảnh báo thiếu; lưu trữ lịch sử tiêu hao

14. **Phân tích và báo cáo:** tạo báo cáo doanh thu, giờ cao điểm, món bán chạy; phân tích chậm trễ và tắc nghẽn vận hành; hỗ trợ xuất dữ liệu

15. **Quản trị và bảo mật:** triển khai RBAC; duy trì audit log cho các hành vi nhạy cảm.

1.5 Yêu cầu phi chức năng

Các yêu cầu phi chức năng được xem là tiêu chí thành công của hệ thống bao gồm:

- **Hiệu năng và độ trễ:** thời gian phản hồi cho thao tác đặt món và truyền đơn đến bếp được yêu cầu nhỏ nhằm đáp ứng điều kiện giờ cao điểm.
- **Độ tin cậy và tính sẵn sàng:** hệ thống được yêu cầu vận hành ổn định trong ca làm việc và hạn chế gián đoạn dịch vụ.
- **Tính nhất quán dữ liệu:** trạng thái đơn hàng, bàn và thanh toán được yêu cầu đồng bộ giữa front-of-house và back-of-house.
- **Tính khả mở:** khả năng mở rộng theo số lượng bàn, thiết bị và (trong định hướng dài hạn) theo nhiều cơ sở vận hành.
- **Bảo mật:** phân quyền chặt chẽ đối với các thao tác nhạy cảm; nhật ký kiểm toán được duy trì để truy vết.

- **Khả năng bảo trì:** thay đổi thực đơn, quy tắc khuyến mãi và luồng điều phối bếp được yêu cầu thực hiện với tác động tối thiểu lên các mô-đun còn lại.
- **Khả năng quan sát:** log và chỉ số vận hành được yêu cầu để hỗ trợ phát hiện lỗi và nhận diện điểm nghẽn.
- **Tính khả dụng (usability):** giao diện thao tác được yêu cầu tối giản nhằm phù hợp với bối cảnh vận hành có cường độ cao.

1.6 Ràng buộc và giả định

Trong phạm vi bài toán được mô tả, các ràng buộc và giả định sau được áp dụng:

- Dữ liệu tiêu hao tồn kho được cập nhật thủ công; cơ chế tự động trừ kho nâng cao không được xem là bắt buộc trong phạm vi hiện tại.
- Cơ chế thông báo khách (SMS/email) được xem như khả năng tích hợp theo mức độ triển khai.
- Luồng xử lý đặt món–điều phối bếp–thanh toán được xem là trục nghiệp vụ chính; các năng lực phân tích nâng cao được xem là phần mở rộng.

1.7 Tiêu chí thành công

Các tiêu chí thành công mang tính định hướng đánh giá hệ thống được đề xuất như sau:

- Độ trễ truyền đơn từ khối phục vụ đến bếp được giảm thiểu.
- Tỷ lệ sai sót do thiếu/nhầm thông tin tùy biến trong đơn hàng được giảm.
- Thời gian ra món (ticket time) được cải thiện và tỷ lệ đúng hạn theo mục tiêu thời gian được nâng cao.
- Báo cáo vận hành đủ chi tiết để xác định giờ cao điểm và món bán chạy.
- Việc mở rộng quy tắc ưu tiên bếp hoặc trạng thái xử lý được thực hiện với thay đổi tối thiểu lên các mô-đun khác, qua đó phản ánh tính mô-đun và khả năng mở rộng kiến trúc.

2 Use Case Scenarios - System Modeling

Các kịch bản sử dụng (Use Case Scenarios) được sử dụng để mô tả có cấu trúc các tương tác giữa tác nhân và hệ thống trong các tình huống nghiệp vụ điển hình. Trên cơ sở đó, các luồng dữ liệu, điểm đồng bộ trạng thái, cũng như các ràng buộc vận hành (ví dụ: độ trễ, khả dụng, tính nhất quán) được làm rõ một cách rõ ràng theo quy trình. Hơn nữa, khi hệ thống được định hướng theo kiến trúc phân tán (ví dụ: Microservices), các Use Case đóng vai trò như một mô tả ranh giới domain, điểm tích hợp (integration points), và các sự kiện/trigger cần được phát sinh hoặc tiêu thụ. Do vậy, các Use Case được xem là đầu vào quan trọng để xác định module/component boundary, cơ chế giao tiếp (sync/async), và các trade-off kiến trúc liên quan.

UC1	**Xem menu (View)**
Use case	View Menu
Mục tiêu	Nhân viên xem menu theo danh mục, trạng thái món, giá hiện tại
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Hệ thống POS/Menu service
Tiền điều kiện	1. Người dùng đã đăng nhập 2. Thiết bị có kết nối mạng (hoặc có cache menu offline) 3. Menu đã được cấu hình và đang Active
Kích hoạt	Nhân viên mở màn hình Menu

Luồng chính	1. Nhân viên chọn module Menu 2. Hệ thống tải menu (danh mục, món, giá, ảnh, mô tả, option, trạng thái còn/hết) 3. Hệ thống hiển thị danh sách món theo Category (Khai vị/Món chính/Đồ uống...) 4. Nhân viên có thể: • Tìm kiếm theo tên/mã món • Lọc theo category, best-seller, đang khuyến mãi • Xem chi tiết món (mô tả, dị ứng, option) 5. Hệ thống cập nhật trạng thái "Còn/Hết" theo tồn kho hoặc cấu hình bếp
Luồng thay thế / ngoại lệ	• A1: Mất mạng → hệ thống hiển thị menu cache gần nhất + cảnh báo "Menu có thể không mới nhất" • A2: Menu trống/chưa publish → hiển thị thông báo và hướng dẫn liên hệ manager • A3: Món bị disable → món vẫn có thể hiện mờ hoặc ẩn tùy cấu hình
Hậu điều kiện	• Menu hiển thị thành công; không thay đổi dữ liệu nghiệp vụ
Quy tắc / dữ liệu	• Giá hiển thị là giá hiệu lực hiện tại (có thể đã áp dụng khuyến mãi) • Món Out of stock không thể chọn để order

UC2	**Thực hiện order (Create)**
Use case	Create Order
Mục tiêu	Server tạo order cho bàn hoặc khách mang đi và gửi vào hệ thống
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Kitchen system, Inventory (nếu trừ tồn), Billing (Có thể có tùy theo dev)
Tiền điều kiện	1. Đã đăng nhập 2. Có bàn hoặc chọn loại order (takeaway/delivery) 3. Menu có sẵn
Kích hoạt	Nhân viên bấm New Order hoặc chọn bàn → Order

Luồng chính	1. Nhân viên chọn bàn hoặc loại đơn (dine-in/takeaway) 2. Hệ thống tạo Order draft với mã tạm 3. Nhân viên chọn món từ menu, nhập số lượng 4. Hệ thống tính tạm tính (subtotal) theo giá hiện hành, hiển thị thời gian tạo 5. Nhân viên xác nhận "Gửi bếp/Place order" 6. Hệ thống: • Validate món còn bán / còn nguyên liệu (nếu có kiểm tra) • Gán số order, trạng thái New • Ghi nhận người tạo, thời điểm 7. Hệ thống đẩy order sang Kitchen system theo prep station 8. Hệ thống hiển thị order trong danh sách theo bàn
Luồng thay thế / ngoại lệ	• A1: Món hết hàng khi gửi → hệ thống báo lỗi, gợi ý món thay thế hoặc cho phép xóa món đó • A2: Order gửi thất bại do mạng → chuyển sang hàng đợi sync, hiển thị "Pending sync", tự retry • A3: 2 người order cùng một bàn → merge order
Hậu điều kiện	• Order được tạo và sẵn sàng cho bếp xử lý
Quy tắc / dữ liệu	• Order phải có ít nhất 1 món • Audit log: tạo order (ai, lúc nào, thiết bị nào)

UC3	**Note Order (Customize)**
Use case	Customize Order Item
Mục tiêu	Ghi nhận yêu cầu đặc biệt, dị ứng, combo, mức chín, thay thế nguyên liệu...
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Server
Tiền điều kiện	1. Có order draft hoặc order đang cho phép chỉnh 2. Món có option/cấu hình note
Kích hoạt	Đang trong quá trình new order hoặc chỉnh sửa ở danh sách order
Luồng chính	1. Nhân viên chọn line-item (món) cần tùy chỉnh 2. Hệ thống hiển thị: • Option có cấu trúc (size, topping, ...) • Note tự do • Allergy alert - Dị ứng 3. Nhân viên chọn option và nhập note 4. Hệ thống hiển thị preview món sau tùy chỉnh + chênh lệch giá (nếu topping tính phí) 5. Nhân viên lưu 6. Khi gửi bếp, hệ thống truyền đầy đủ option/note đến Kitchen system

Luồng thay thế / ngoại lệ	• A1: Note vượt giới hạn ký tự → hệ thống yêu cầu rút gọn • A2: Option xung đột (vd: "no ice" nhưng chọn "extra ice") → hệ thống cảnh báo và yêu cầu chọn lại • A3: Allergy flag bật → hệ thống yêu cầu xác nhận thêm ("Bạn đã xác nhận với khách?")
Hậu điều kiện	• Lưu tùy chỉnh
Quy tắc / dữ liệu	• Một số note có thể bị chặn theo policy (vd: không cho "free topping" nếu không có quyền)

UC4	**Quản lý món ăn trong menu**
Use case	Menu Item CRUD
Mục tiêu	Admin/Manager thêm, sửa, xóa (hoặc disable) món
Actors chính	Administrator/Manager
Actors phụ	Audit/Logging, Sync service tới thiết bị
Tiền điều kiện	1. Người dùng có quyền "Menu:Manage" 2. Có danh mục menu sẵn (hoặc tạo danh mục trước)
Kích hoạt	Admin mở "Menu Management"

Luồng chính (Create)	1. Admin chọn "Add item" 2. Nhập thông tin: tên, category, mô tả, ảnh, prep station, tax class, tags dị ứng, option set 3. Nhập giá mặc định 4. Chọn trạng thái: Active/Inactive 5. Lưu 6. Hệ thống validate dữ liệu và tạo món mới 7. Hệ thống publish thay đổi đến các hệ thống khác
Luồng chính (Update)	1. Admin chọn món → "Edit" 2. Sửa thuộc tính, giá 3. Lưu và publish
Luồng chính (Delete/Disable)	1. Admin chọn món → "Delete" hoặc "Disable" 2. Hệ thống kiểm tra ràng buộc: • Món có trong order đang mở? • Món có trong promo/combo đang active? 3. Nếu được phép → xóa hoặc set Inactive 4. Publish

Luồng thay thế / ngoại lệ	• A1: Không cho xóa hard-delete vì có lịch sử → hệ thống chỉ cho "Disable" • A2: Thiếu trường bắt buộc → báo lỗi cụ thể • A3: Xung đột đồng bộ (2 admin sửa) → cơ chế versioning/last-write-wins + cảnh báo
Hậu điều kiện	• Menu được cập nhật; audit log ghi nhận

UC5	**Thực hiện khuyến mãi trên menu**
Use case	Manage Promotions
Mục tiêu	Thiết lập promo theo món/combo/khung giờ/điều kiện
Actors chính	Manager/Admin
Tiền điều kiện	1. Có quyền "Promo:Manage" 2. Menu items đã tồn tại
Kích hoạt	Mở "Promotions"

Luồng chính	1. Manager chọn "Create Promotion" 2. Chọn loại: • Giảm % theo món • Giảm số tiền cố định • Combo set price • Buy X Get Y 3. Chọn danh sách món áp dụng và điều kiện (giờ, ngày, số lượng, nhóm khách...) 4. Đặt thời gian hiệu lực và ưu tiên (priority) 5. Lưu 6. Hệ thống validate xung đột promo và tính thử (simulation) 7. Publish promo đến POS
Luồng thay thế / ngoại lệ	• A1: Promo xung đột → hệ thống yêu cầu đặt priority hoặc không cho chồng • A2: Combo thiếu món thành phần → báo lỗi • A3: Promo hết hạn → tự động disable
Hậu điều kiện	• Promo hoạt động; giá hiển thị/áp vào bill theo rule

UC6	**Truyền order đến nhà bếp**
Use case	Send Order to Kitchen (Realtime)
Mục tiêu	Order mới xuất hiện trên Kitchen system gần realtime, đúng station

Actors chính	Server (người tạo), Kitchen team (người nhận)
Actors phụ	Message broker/Sync service (Nếu có)
Tiền điều kiện	1. Order đã "Placed" 2. Kitchen system online hoặc có cơ chế nhận lại khi online
Kích hoạt	Order được gửi/được update
Luồng chính	1. POS gửi event "OrderPlaced/OrderUpdated" 2. Hệ thống phân tách ticket theo prep station (grill/salad/bar...) 3. Kitchen system nhận event và hiển thị ticket mới 4. Kitchen phát âm thanh/notify theo cấu hình 5. Hệ thống ghi log thời gian "sent-to-kitchen"
Luồng thay thế / ngoại lệ	• A1: Kitchen system offline → lưu hàng đợi; khi Kitchen system online thì replay • A2: Duplicate event → Kitchen system idempotent - nhận diện "đây là event đã xử lý rồi" và bỏ qua/ghi đè. • A3: Station mapping thiếu → đưa vào "Unassigned" và cảnh báo manager
Hậu điều kiện	• Bếp nhìn thấy order và có thể bắt đầu xử lý

UC7	**Hiển thị danh sách order (Kitchen View)**
Use case	Kitchen Order Queue View
Mục tiêu	Bếp xem hàng đợi theo thời gian, theo loại món, theo station

Actors chính	Chef/Kitchen staff
Tiền điều kiện	1. Đăng nhập vào kitchen system 2. Có order đang chờ
Kích hoạt	Mở màn hình "Queue"
Luồng chính	1. Nhân viên bếp chọn chế độ xem: All / Station / Category 2. Hệ thống hiển thị danh sách ticket theo thứ tự (FIFO hoặc theo SLA - Thời gian tối đa có thể để hoàn thành 1 món) 3. Mỗi ticket hiển thị: bàn, thời gian vào bếp, danh sách món, note dị ứng, trạng thái 4. Cho phép filter: "New", "In progress", "Near deadline" 5. Auto-refresh realtime
Luồng thay thế / ngoại lệ	• A1: Quá nhiều ticket → phân trang/virtual list • A2: Lỗi sync → hiển thị trạng thái "Last updated at ..."
Hậu điều kiện	Không đổi dữ liệu (chỉ view)

UC8	**Quản lý order trong nhà bếp (Notify + Deadline)**
Use case	Kitchen Order Monitoring & Alerts
Mục tiêu	Cảnh báo order mới và order gần quá SLA/deadline
Actors chính	Kitchen lead/Chef

Tiền điều kiện	1. SLA time per item/station được cấu hình (hoặc default) 2. Kitchen system bật notifications
Kích hoạt	Order mới đến hoặc timer đạt ngưỡng cảnh báo
Luồng chính	1. Khi ticket mới đến, Kitchen system phát thông báo âm thanh + badge "New" 2. Hệ thống bắt đầu đếm timer từ "sent-to-kitchen" hoặc "start cooking" tùy rule 3. Khi gần deadline (vd: 80% SLA), ticket đổi màu và đưa lên top 4. Khi quá deadline, Kitchen system hiển thị cảnh báo mạnh hơn (màu đỏ/đẩy lên đầu) 5. Kitchen lead có thể mở ticket để xem (station nào đang chậm)
Luồng thay thế / ngoại lệ	• A1: SLA chưa cấu hình → dùng default (ví dụ 15 phút) và cảnh báo "SLA default" • A2: Ticket bị pause (do chờ khách) → cho phép "Hold" để dừng timer (nếu có quyền)
Hậu điều kiện	Trạng thái cảnh báo cập nhật

UC9	**Cập nhật trạng thái order (Kitchen Update)**
Use case	Update Cooking Status
Mục tiêu	Chef cập nhật trạng thái theo tiến trình: Bắt đầu làm → Đang nấu → Đã xong

Actors chính	Chef/Kitchen staff
Actors phụ	Server (nhận thông báo món xong), Front of House display
Tiền điều kiện	1. Ticket tồn tại và thuộc station của chef 2. Người dùng có quyền "Kitchen:UpdateStatus"
Kích hoạt	Chef thao tác trên ticket
Luồng chính	1. Chef mở ticket 2. Bấm "Start" → trạng thái ticket/item chuyển sang In progress 3. (Option) bấm "Cooking"/step tiếp theo nếu hệ thống có nhiều bước 4. Khi hoàn tất, bấm "Done" 5. Hệ thống: • ghi thời gian start/finish • cập nhật Kitchen system và gửi event về Front of House để server biết món ra 6. Front of House hiển thị "Ready to serve"
Luồng thay thế / ngoại lệ	• A1: Chef bấm nhầm "Done" → nếu policy cho phép, có "Undo" trong X giây + audit
Hậu điều kiện	• Trạng thái món và ticket cập nhật; dữ liệu phục vụ báo cáo hiệu suất

UC10	**Quản lý bàn ghế (Table Management)**

Use case	Manage Tables
Mục tiêu	Quản lý sơ đồ bàn, trạng thái bàn trống/đang phục vụ/đang chờ dọn,...
Actors chính	Host/Server/Manager
Tiền điều kiện	1. Có floor plan và danh sách bàn 2. Có quyền "Table:View" (server) hoặc "Table:Manage" (manager)
Kích hoạt	Mở "Floor plan/Tables"
Luồng chính	1. Host xem bản đồ bàn theo khu vực 2. Hệ thống hiển thị trạng thái: • Available / Seated / Ordering / Serving / Check requested / Dirty 3. Khi xếp khách: • Host chọn bàn → "Seat party" • nhập số khách, tên (tùy) • hệ thống set bàn sang Seated và bắt đầu timer 4. Server có thể mở bàn để xem order hiện có, trạng thái món, tình trạng hóa đơn 5. Manager có thể gộp/tách bàn, đổi bàn (move table), khóa bàn

Luồng thay thế / ngoại lệ	• A1: Bàn đã có khách nhưng host seat nhầm → hệ thống cảnh báo + yêu cầu quyền manager • A2: Move table khi đã có order → hệ thống chuyển toàn bộ order & bill mapping sang bàn mới
Hậu điều kiện	• Trạng thái bàn cập nhật và phản ánh trên toàn hệ thống

UC11	**Xử lý đặt bàn và hàng đợi (Reservation & Queue)**
Use case	Reservation & Waitlist
Mục tiêu	Đặt bàn theo lịch, quản lý hàng đợi, gửi thông báo cho khách
Actors chính	System
Actors phụ	Customer (nhận thông báo), Messaging service (SMS/Zalo/Email)
Tiền điều kiện	1. Có cấu hình giờ mở cửa, sức chứa, bàn 2. Tích hợp kênh nhắn tin (nếu có)
Kích hoạt	Host tạo reservation hoặc thêm waitlist
Luồng chính (Reservation)	1. Host chọn "New reservation" 2. Nhập: tên, SĐT, số khách, ngày giờ, ghi chú 3. Hệ thống gợi ý bàn phù hợp và check conflict 4. Host xác nhận 5. Hệ thống tạo reservation trạng thái "Booked" 6. (Tùy) Gửi xác nhận cho khách

Luồng chính (Waitlist)	1. Host chọn "Add to waitlist" 2. Nhập thông tin khách + party size 3. Hệ thống ước tính thời gian chờ (dựa trên turnover lịch sử + bàn trống) 4. Khi có bàn trống, host bấm "Notify" 5. Hệ thống gửi thông báo + đặt thời hạn giữ chỗ (hold time) 6. Host seat khách → chuyển waitlist entry thành seated
Luồng thay thế / ngoại lệ	• A1: Khách không đến → đánh dấu no-show; ghi log • A2: Không gửi được SMS → hiển thị lỗi và cho phép gọi tay
Hậu điều kiện	• Reservation/waitlist cập nhật; timeline phục vụ đồng bộ với bàn

UC12	**Quản lý hóa đơn (Billing & Payment)**
Use case	Bill Management & Payment
Mục tiêu	Gom món vào hóa đơn theo bàn, thanh toán đa phương thức, chia bill, tip
Actors chính	Cashier/Server
Actors phụ	Payment gateway, Bank/Wallet, Manager (refund/void)
Tiền điều kiện	1. Bàn có order đã phục vụ hoặc đang phục vụ 2. Có quyền "Bill:Checkout"

Kích hoạt	Chọn bàn → "Checkout/Payment"
Luồng chính	1. Cashier mở hóa đơn của bàn 2. Hệ thống hiển thị danh sách món, trạng thái phục vụ, subtotal, tax, discount, tip 3. Nếu khách yêu cầu chia bill: • chọn "Split" • chia theo item / theo người / theo % 4. Chọn phương thức thanh toán: thẻ / chuyển khoản / ví / tiền mặt 5. Nhập/nhận số tiền, tip 6. Hệ thống xử lý thanh toán (gọi gateway nếu online) 7. Nếu thành công: • hóa đơn trạng thái "Paid" • in/ gửi hóa đơn điện tử (tùy) • bàn chuyển trạng thái "Dirty/Cleaning" 8. Lưu audit
Luồng thay thế / ngoại lệ	• A1: Gateway fail → cho retry hoặc đổi phương thức • A2: Thanh toán một phần (partial) → hóa đơn "Partially paid", còn balance • A3: Refund/Void → yêu cầu quyền manager + lý do + audit log
Hậu điều kiện	• Doanh thu ghi nhận, dữ liệu báo cáo cập nhật

UC13	**Quản lý số lượng hàng hóa (Inventory)**
Use case	Ingredient Usage & Low-stock Alerts
Mục tiêu	Nhân viên nhập lượng sử dụng nguyên liệu, cảnh báo thấp, dự đoán reorder
Actors chính	Kitchen staff/Manager
Actors phụ	Reporting/Forecasting service
Tiền điều kiện	1. Có danh mục nguyên liệu + mức min threshold 2. Có mapping món → nguyên liệu (nếu muốn tự động trừ)
Kích hoạt	Nhập usage hoặc hệ thống tự trừ theo order
Luồng chính (Manual usage)	1. Nhân viên mở module Inventory → "Record usage" 2. Chọn nguyên liệu, nhập số lượng dùng và đơn vị 3. Chọn lý do (prep/spoilage/adjustment) 4. Lưu 5. Hệ thống cập nhật tồn kho, ghi audit
Luồng chính (Low-stock alert)	1. Khi tồn < threshold, hệ thống gửi cảnh báo tới manager 2. Manager xem danh sách cần reorder
Luồng chính (Forecast reorder)	1. Hệ thống tổng hợp lịch sử usage/sales 2. Đưa ra dự báo lượng cần đặt cho kỳ tới (ngày/tuần) 3. Manager xuất danh sách reorder

Luồng thay thế / ngoại lệ	• A1: Nhập sai đơn vị → hệ thống yêu cầu quy đổi hoặc từ chối • A2: Không đủ quyền điều chỉnh tồn → chỉ manager được chỉnh
Hậu điều kiện	• Tồn kho cập nhật; cảnh báo & dự báo sẵn sàng

UC14	**Phân tích và báo cáo (Analytics & Reporting)**
Use case	Sales & Operations Reporting
Mục tiêu	Báo cáo doanh thu, giờ cao điểm, best-seller, bottleneck, hiệu suất nhân sự; xuất báo cáo
Actors chính	Administrator/Manager
Tiền điều kiện	1. Có quyền "Reports:View" 2. Có dữ liệu giao dịch
Kích hoạt	Mở module "Reports"

Luồng chính	1. Manager chọn loại báo cáo: Sales, Menu performance, Labor, Kitchen SLA, Table turnover 2. Chọn khoảng thời gian, chi nhánh, ca làm 3. Hệ thống hiển thị dashboard: • doanh thu theo giờ/ngày • top món bán chạy • ticket time trung bình theo station • tỷ lệ hủy/void/refund 4. Manager chọn "Export" (CSV/PDF) 5. Hệ thống tạo file và cho tải về (hoặc gửi email)
Luồng thay thế / ngoại lệ	• A1: Khoảng thời gian quá lớn → hệ thống chạy nền và thông báo khi xong • A2: Không có dữ liệu → hiển thị trạng thái empty
Hậu điều kiện	• Báo cáo được xem/xuất; không ảnh hưởng vận hành

UC15	**Kiểm soát an toàn và quyền truy cập (RBAC - Kiểm soát truy cập theo vai trò + Audit logs)**
Use case	Access Control & Audit Logs
Mục tiêu	Phân quyền theo role; audit các hành động nhạy cảm (hủy đơn, hoàn tiền...)
Actors chính	Administrator
Actors phụ	Manager (xử lý nghiệp vụ nhạy cảm), Auditor

Tiền điều kiện	1. Hệ thống có danh sách role: managers, servers, chefs, cashiers 2. Người dùng được gán role
Kích hoạt	User thao tác chức năng cần quyền hoặc admin xem audit
Luồng chính (RBAC) - Role-based access control:	1. User thực hiện hành động (vd: refund) 2. Hệ thống kiểm tra quyền theo role + policy 3. Nếu đủ quyền → cho phép thao tác 4. Nếu không đủ → chặn và hiển thị thông báo "Access denied"
Luồng chính (Audit log)	1. Khi hành động nhạy cảm xảy ra (void/refund/discount override/price change): 2. Hệ thống ghi audit: userId, role, action, before/after, timestamp, reason, device 3. Admin mở "Audit logs", filter theo thời gian/user/action 4. Admin export log nếu cần
Luồng thay thế / ngoại lệ	• A1: Manager override: server không có quyền → yêu cầu manager nhập PIN/approve • A2: Audit storage lỗi → vẫn cho phép/hoặc chặn tùy mức độ nghiêm trọng
Hậu điều kiện	• Quyền truy cập được đảm bảo; truy vết đầy đủ

3 So sánh và lựa chọn phong cách kiến trúc phù hợp (Compare and choose suitable architecture styles)

Việc chọn architecture style cho IRMS được xem như một bài toán trade-off giữa các *architecture characteristics* (non-functional requirements). IRMS không chỉ có nhiều chức năng (menu, order, kitchen workflow, table/reservation, billing, inventory, reporting) mà còn yêu cầu vận hành trơn chu giữa front-of-house và kitchen, tức là phải giảm delay/error và đồng bộ trạng thái theo thời gian gần realtime. Vì vậy, các tiêu chí như performance, availability/reliability, scalability, maintainability/extensibility và security được ưu tiên khi so sánh các style.

3.1 Modular Monolith.

Theo ma trận đánh giá, Modular Monolith có điểm mạnh lớn nhất ở simplicity và cost thấp. Với bối cảnh đó, cách tiếp cận này giúp nhóm build nhanh, debug dễ và hạn chế overhead vận hành (deploy 1 lần thay vì nhiều service). Tuy nhiên, worksheet cũng chỉ ra monolith bị hạn chế ở scalability, elasticity và fault-tolerance. Đối với IRMS, giờ cao điểm có thể tạo bottleneck ở các luồng như "place order → send to kitchen → update status → checkout". Nếu toàn bộ hệ thống chạy chung một khối, khi một module (ví dụ kitchen queue hoặc payment) gặp sự cố hoặc bị quá tải, hiệu ứng lan truyền có thể làm giảm availability của toàn hệ thống. Đây là trade-off quan trọng cần cân nhắc.

3.2 Microservices.

Microservices được đánh giá cao nhất về maintainability, testability, deployability, scalability, fault-tolerance và evolvability. Điều này phù hợp với IRMS vì các module có vòng đời thay đổi khác nhau: menu/promotion thường xuyên update, kitchen workflow cần tối ưu liên tục, billing/payment có tính nhạy cảm và hay phụ thuộc third-party, còn reporting có thể xử lý theo batch. Khi tách thành các component/service độc lập, việc scale và deploy từng phần sẽ chủ động hơn (ví dụ scale riêng kitchen hoặc ordering trong giờ cao điểm), đồng thời việc refactor hoặc nâng cấp cũng ít ảnh hưởng dây chuyền.

Trade-off ở đây là chi phí triển khai và vận hành tăng (nhiều service, nhiều API call giữa service, cần cơ chế quan sát/monitoring tốt), tức là overhead sẽ lớn hơn so với monolith.

3.3 Event-driven.

Event-driven architecture được worksheet đánh giá nổi bật ở responsiveness, fault-tolerance và evolvability. Với IRMS, nhiều tình huống tự nhiên đã mang tính sự kiện: OrderPlaced/OrderUpdated, ticket gần deadline, món Done/Ready-to-serve, payment success/fail, low-stock alert, audit log... Nếu các tương tác này được handle bằng event (publish/subscribe) thay vì gọi đồng bộ trực tiếp, các component sẽ giảm coupling, giảm rủi ro chờ đợi khi một service chậm, và dễ mở rộng thêm hành vi (ví dụ thêm module analytics lắng nghe sự kiện để tính SLA) mà không cần sửa code các module hiện có. Trade-off là hệ thống phức tạp hơn ở phần consistency và xử lý lỗi: cần thiết kế idempotency (tránh duplicate event), cơ chế retry/replay khi hệ thống bếp offline, và chấp nhận một mức eventual consistency cho một số trạng thái.

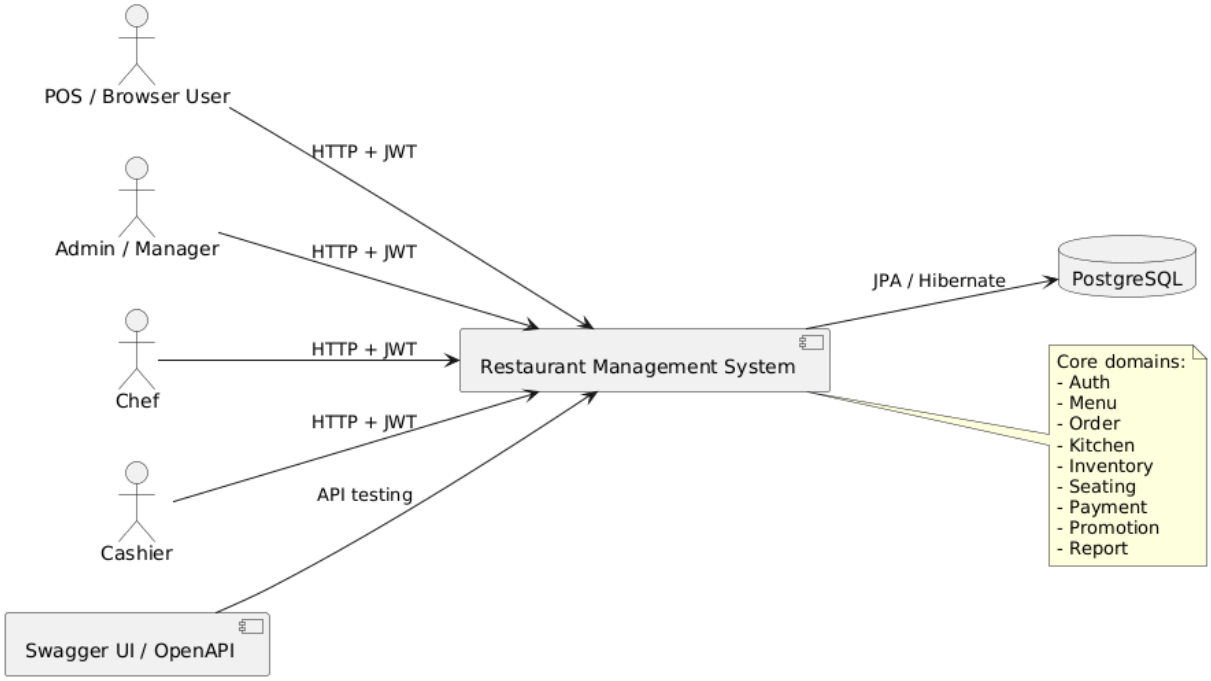
3.4 Kết luận lựa chọn.

Từ các phân tích trên, báo cáo này định hướng lựa chọn kiến trúc theo hướng kết hợp: dùng phân rã theo các "architecture quantum" có thể deploy độc lập (phù hợp với microservices) cho các domain chính như Ordering, Kitchen, Billing, Inventory, Reporting; đồng thời các luồng liên module cần realtime và ít coupling (đặc biệt là Ordering $\leftrightarrow$ Kitchen theo UC6–UC9) sẽ được thiết kế theo event-driven để tăng responsiveness và fault-tolerance. Khi thiết kế các component, anti-pattern "entity trap" cần được tránh (không tạo các lớp/manager khổng lồ theo từng entity), thay vào đó ranh giới module/component nên bám theo chức năng và workflow thực tế (ví dụ: kitchen ticket workflow + state machine) nhằm giữ cohesion cao và hỗ trợ maintain/extensibility theo yêu cầu của hệ thống.

4 Xây dựng kiến trúc phần mềm cho hệ thống

4.1 Module Views

4.1.1 Context View



Hình 1: Context View của hệ thống

Sơ đồ Context View mô tả mối quan hệ tổng quát giữa hệ thống quản lý nhà hàng và các tác nhân bên ngoài. Hệ thống đóng vai trò trung tâm, tiếp nhận yêu cầu từ nhiều nhóm người dùng khác nhau thông qua giao thức HTTP kết hợp cơ chế xác thực JWT.

POS / Browser User: Người dùng thông thường sử dụng hệ thống thông qua giao diện POS hoặc trình duyệt web để thực hiện các chức năng như xem thực đơn, đặt món và thanh toán.

Admin / Manager: Quản trị viên hoặc quản lý sử dụng hệ thống để quản lý menu, khuyến mãi, báo cáo doanh thu và giám sát hoạt động nhà hàng.

Chef: Đầu bếp nhận thông tin đơn hàng từ hệ thống để xử lý và cập nhật trạng thái món ăn.

Cashier: Thu ngân sử dụng hệ thống để xử lý thanh toán và xác nhận giao dịch.

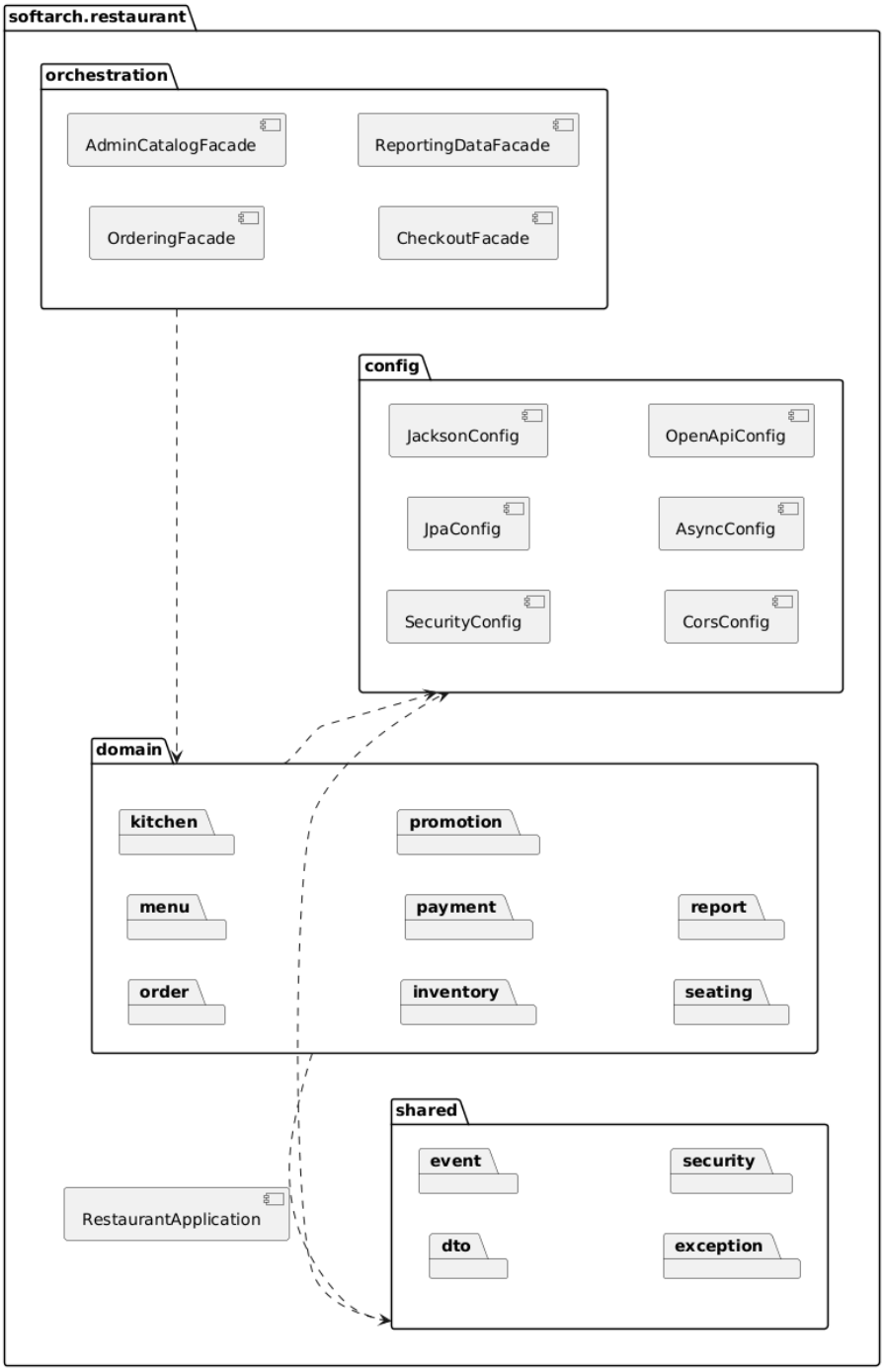
Restaurant Management System: Là hệ thống trung tâm chứa các domain nghiệp vụ chính bao gồm: *Auth, Menu, Order, Kitchen, Inventory, Seating, Payment, Promotion, Report.*

PostgreSQL: Được sử dụng làm hệ quản trị cơ sở dữ liệu nhằm lưu trữ dữ liệu nghiệp vụ của hệ thống thông qua cơ chế JPA/Hibernate.

Swagger UI / OpenAPI: Hỗ trợ kiểm thử và tài liệu hóa RESTful API trong quá trình phát triển và tích hợp hệ thống.

Luồng giao tiếp: Người dùng gửi request → Restaurant Management System → PostgreSQL.

4.1.2 Package View



Hình 2: Package View của hệ thống

Sơ đồ Package View mô tả cấu trúc tổ chức mã nguồn của hệ thống theo các package chức năng nhằm tăng tính phân tách trách nhiệm, khả năng mở rộng và bảo trì.

Package config: Chứa các thành phần cấu hình hệ thống như: *SecurityConfig,*

JpaConfig, JacksonConfig, CorsConfig, AsyncConfig, OpenApiConfig. Các package này hỗ trợ cấu hình bảo mật, ORM, serialization, CORS, xử lý bất đồng bộ và tài liệu API.

Package shared: Chứa các thành phần dùng chung trong toàn hệ thống bao gồm: *dto, event, exception, security.* Các thành phần này giúp tái sử dụng mã nguồn và giảm trùng lặp logic.

Package orchestration: Chứa các facade chịu trách nhiệm điều phối luồng nghiệp vụ giữa nhiều domain khác nhau như: *OrderingFacade, CheckoutFacade, AdminCatalogFacade, ReportingDataFacade.*

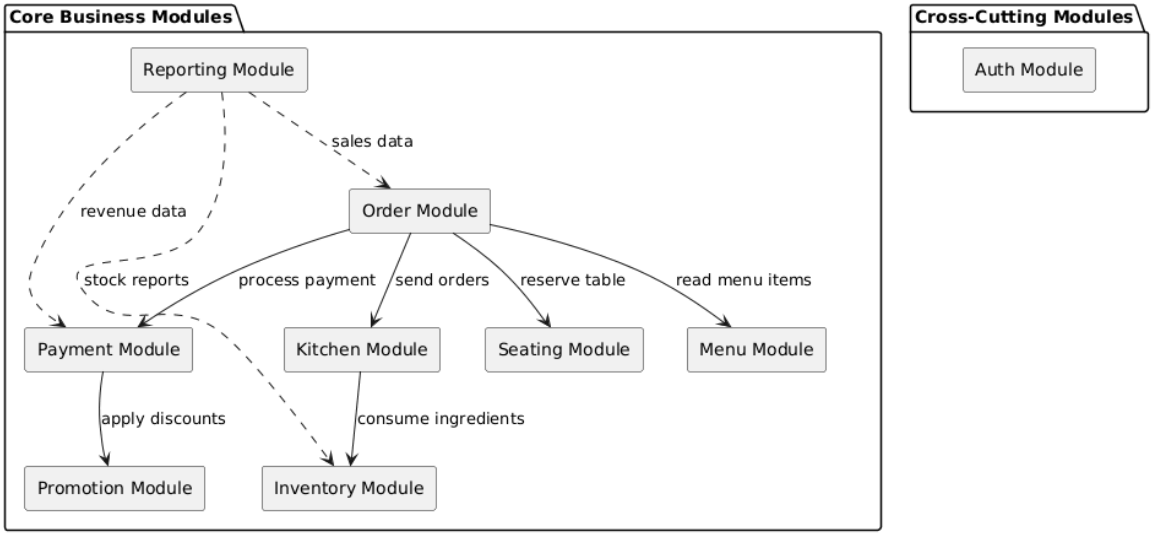
Package domain: Chứa các domain nghiệp vụ chính của hệ thống: *order, menu, kitchen, inventory, payment, promotion, seating, report.* Mỗi domain được tổ chức độc lập nhằm đảm bảo tính module hóa và giảm coupling giữa các thành phần.

RestaurantApplication: Là điểm khởi chạy chính của toàn bộ hệ thống Spring Boot.

Quan hệ phụ thuộc:

- Package *shared* phụ thuộc vào *config* để sử dụng các cấu hình hệ thống.
- Package *orchestration* phụ thuộc vào *domain* để điều phối nghiệp vụ.
- Package *domain* sử dụng các thành phần dùng chung từ *shared.*
- Package *domain* đồng thời phụ thuộc vào *config* cho các cấu hình hạ tầng.

4.1.3 Module View



Hình 3: Module View của hệ thống

Sơ đồ Module View mô tả cấu trúc phân rã hệ thống theo các business module chính nhằm tăng tính độc lập, khả năng mở rộng và tái sử dụng.

Hệ thống được chia thành hai nhóm module chính:

Core Business Modules: Bao gồm các module nghiệp vụ cốt lõi của hệ thống:

- *Menu Module*: quản lý thực đơn và thông tin món ăn.
- *Order Module*: xử lý đặt món và quản lý đơn hàng.
- *Kitchen Module*: quản lý hàng chờ chế biến món ăn.
- *Payment Module*: xử lý thanh toán và giao dịch.
- *Inventory Module*: quản lý nguyên liệu và tồn kho.
- *Promotion Module*: quản lý khuyến mãi và giảm giá.
- *Seating Module*: quản lý bàn và đặt chỗ.
- *Reporting Module*: tổng hợp báo cáo và thống kê doanh thu.

Cross-Cutting Modules: Bao gồm các module hỗ trợ xuyên suốt toàn hệ thống:

- *Auth Module*: cung cấp cơ chế xác thực và phân quyền người dùng thông qua JWT Security.

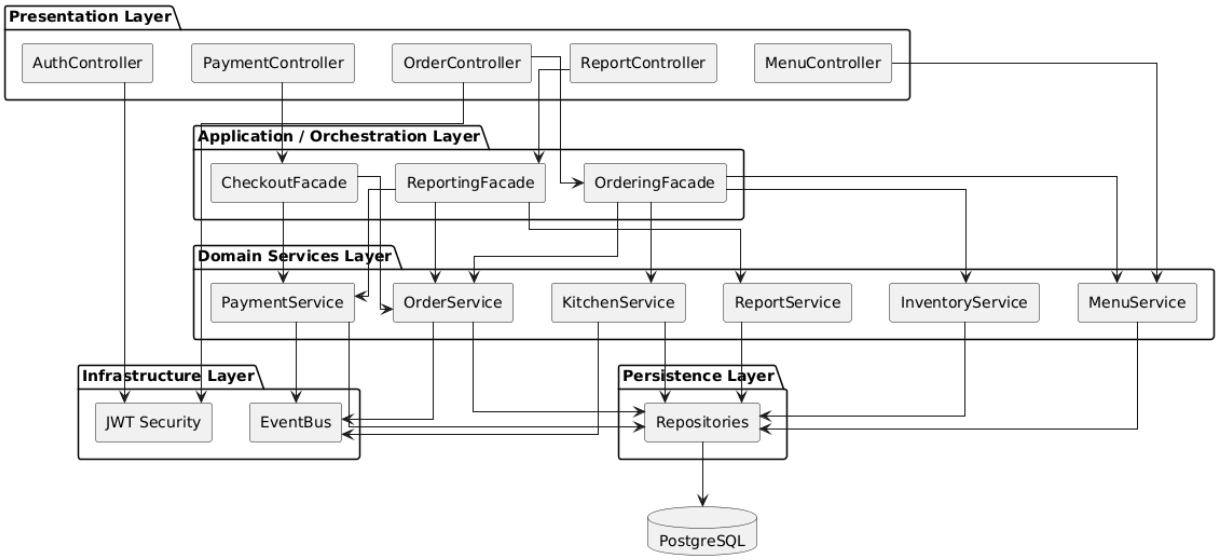
Quan hệ giữa các module:

- *Order Module* sử dụng dữ liệu từ *Menu Module* để tạo đơn hàng.
- *Order Module* tương tác với *Payment Module* để xử lý thanh toán.
- *Order Module* gửi thông tin đơn hàng tới *Kitchen Module*.
- *Kitchen Module* cập nhật và tiêu thụ nguyên liệu từ *Inventory Module*.
- *Payment Module* áp dụng khuyến mãi từ *Promotion Module*.
- *Order Module* tương tác với *Seating Module* để quản lý đặt bàn.
- *Reporting Module* thu thập dữ liệu từ các module nghiệp vụ nhằm tạo báo cáo doanh thu, thống kê đơn hàng và tình trạng tồn kho.

Sơ đồ Module View giúp mô tả rõ ràng cấu trúc nghiệp vụ của hệ thống và mối quan hệ giữa các business module chính trong kiến trúc phần mềm.

4.2 Component & Connector Views

4.2.1 Component View



Hình 4: Component View của hệ thống

Sơ đồ Component View mô tả cấu trúc thành phần bên trong hệ thống và cách các component được tổ chức theo mô hình phân lớp nhằm tăng khả năng mở rộng, tái sử dụng và bảo trì hệ thống.

Khác với Module View chỉ tập trung vào các business module ở mức khái quát, Component View đi sâu hơn vào các thành phần kỹ thuật cụ thể và trách nhiệm của từng component trong hệ thống.

Hệ thống được chia thành các layer chính như sau:

Presentation Layer: Bao gồm các controller như: *AuthController, OrderController, MenuController, PaymentController, ReportController*. Các controller đóng vai trò là điểm tiếp nhận HTTP request từ client và chuyển tiếp request tới các lớp xử lý nghiệp vụ tương ứng.

Application / Orchestration Layer: Bao gồm các facade như: *OrderingFacade, CheckoutFacade, ReportingFacade*. Lớp này chịu trách nhiệm điều phối các luồng nghiệp vụ phức tạp giữa nhiều domain service khác nhau, giúp giảm coupling giữa Presentation Layer và Domain Layer.

Domain Services Layer: Chứa các service nghiệp vụ chính của hệ thống bao gồm: *OrderService, MenuService, KitchenService, PaymentService, InventoryService, ReportService*. Đây là lớp xử lý business logic cốt lõi của hệ thống.

Persistence Layer: Bao gồm các repository chịu trách nhiệm truy cập và lưu trữ dữ liệu thông qua JPA/Hibernate.

Infrastructure Layer: Bao gồm các thành phần hạ tầng như: *EventBus* và *JWT Security*. Lớp này hỗ trợ xác thực người dùng và giao tiếp bất đồng bộ thông qua cơ chế event-driven.

Database: Hệ thống sử dụng PostgreSQL làm cơ sở dữ liệu trung tâm để lưu trữ toàn bộ dữ liệu nghiệp vụ.

Luồng hoạt động của Component View

Quá trình xử lý request trong hệ thống diễn ra theo các bước sau:

1. Client gửi HTTP request tới hệ thống.
2. JWT Security thực hiện xác thực và phân quyền request trước khi chuyển tiếp tới controller tương ứng.
3. Controller tiếp nhận request:

- Với các tác vụ đơn giản, controller có thể gọi trực tiếp service.
- Với các workflow phức tạp, controller sẽ gọi tới facade để điều phối nghiệp vụ.

4. Facade thực hiện orchestration giữa nhiều domain service khác nhau nhằm xử lý business logic.
5. Domain service thao tác với repository để truy cập hoặc cập nhật dữ liệu trong PostgreSQL.
6. Trong quá trình xử lý, service có thể publish domain events tới EventBus.
7. EventBus thực hiện dispatch event bất đồng bộ tới các listener hoặc service khác nhằm xử lý các tác vụ follow-up.

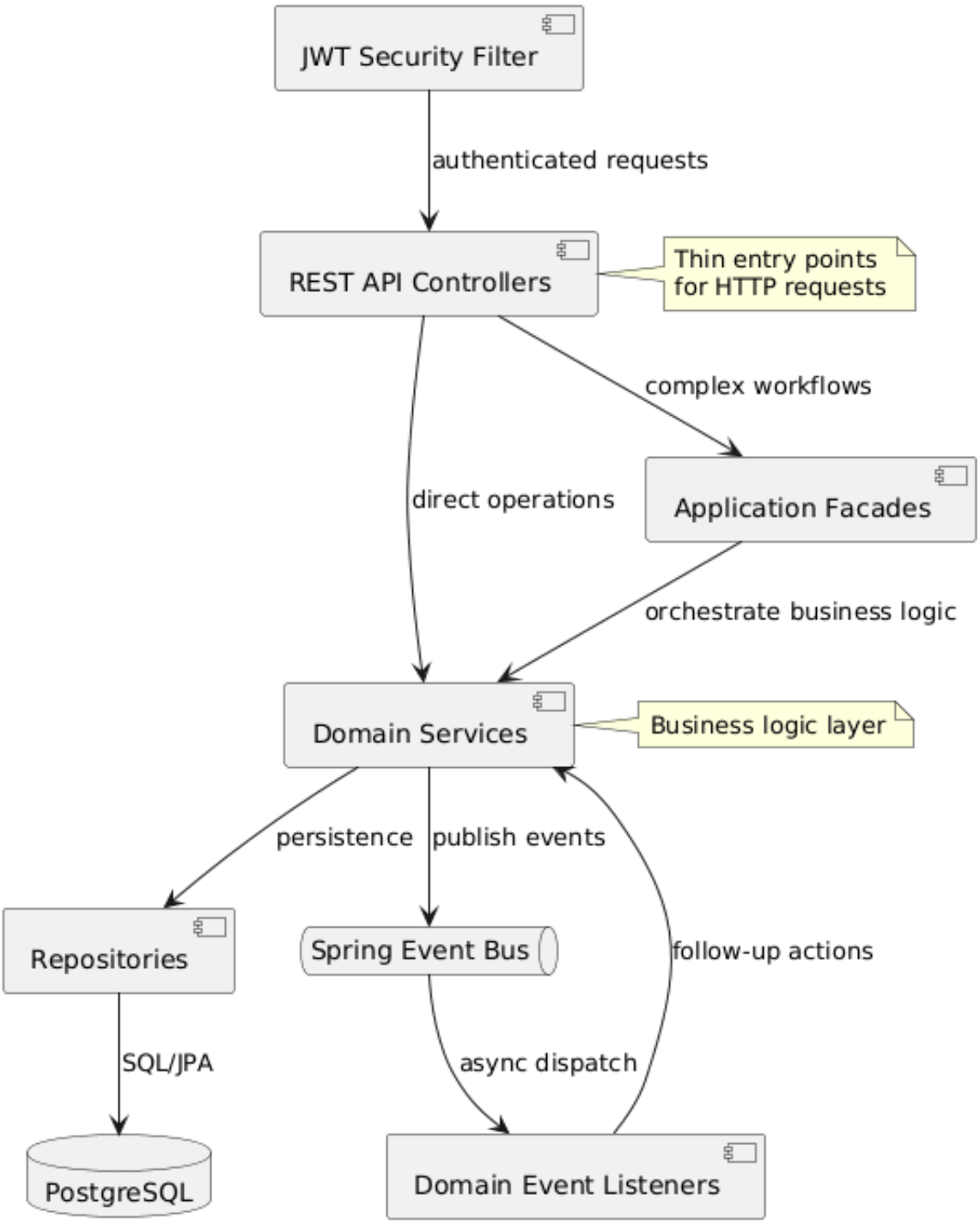
Guideline cho Component View

Khi xây dựng Component View cho hệ thống, cần tuân thủ các nguyên tắc sau:

- Component View phải tập trung mô tả cấu trúc thành phần bên trong hệ thống thay vì business capability ở mức cao.
- Các component nên được tổ chức theo layer nhằm tăng tính phân tách trách nhiệm (Separation of Concerns).
- Controller cần giữ vai trò "thin controller", chỉ tiếp nhận request và chuyển tiếp xử lý xuống các layer bên dưới.
- Các workflow nghiệp vụ phức tạp nên được xử lý thông qua facade hoặc orchestration layer để tránh coupling trực tiếp giữa controller và nhiều service.
- Business logic phải được đặt tại domain services thay vì controller hoặc repository.
- Repository chỉ chịu trách nhiệm persistence và không chứa business logic.
- Các thành phần cross-cutting như authentication, event bus hoặc logging nên được tách riêng vào infrastructure layer.

- Các interaction bất đồng bộ nên được biểu diễn thông qua event-driven connectors như EventBus.
- Không nên đưa quá nhiều implementation detail vào Component View như entity fields hoặc method-level design vì điều này phù hợp hơn với Class Diagram.
- Component View cần đảm bảo người đọc có thể nhanh chóng hiểu:
  - Hệ thống gồm những component nào.
  - Vai trò của từng component.
  - Quan hệ phụ thuộc giữa các component.
  - Luồng xử lý chính của hệ thống.

4.2.2 Component & Connector (C&C) View



Hình 5: Component & Connector View của hệ thống

Sơ đồ Component & Connector (C&C) View mô tả các thành phần runtime của hệ thống và cách các thành phần này giao tiếp với nhau trong quá trình thực thi.

Khác với Component View tập trung vào cấu trúc thành phần, C&C View tập

trung vào:

- Runtime interaction.
- Luồng xử lý request.
- Event-driven communication.
- Quan hệ giao tiếp giữa các component.

Các thành phần chính trong sơ đồ bao gồm:

JWT Security Filter: Thực hiện xác thực và kiểm tra quyền truy cập trước khi request được xử lý bởi hệ thống.

REST API Controllers: Tiếp nhận request HTTP từ client và định tuyến request tới các facade hoặc service tương ứng.

Application Facades: Điều phối các workflow nghiệp vụ phức tạp giữa nhiều domain service khác nhau.

Domain Services: Xử lý business logic chính của hệ thống.

Repositories: Thực hiện persistence và truy vấn dữ liệu từ PostgreSQL.

Spring Event Bus: Cung cấp cơ chế giao tiếp bất đồng bộ giữa các thành phần thông qua event-driven architecture.

Domain Event Listeners: Lắng nghe và xử lý các event được publish từ domain services.

Luồng xử lý của C&C View

Luồng xử lý runtime của hệ thống được thực hiện như sau:

1. Request từ client được xác thực thông qua JWT Security Filter.
2. Request được chuyển tới REST API Controllers.
3. Controller:
    - Gọi trực tiếp domain services cho các thao tác đơn giản.
    - Gọi facade cho các workflow phức tạp cần orchestration.

4. Domain services xử lý business logic và thao tác với repositories để truy cập dữ liệu.
5. Repositories tương tác với PostgreSQL thông qua JPA/Hibernate.
6. Services có thể publish domain events tới Spring Event Bus.
7. EventBus thực hiện dispatch bất đồng bộ tới các listener.
8. Listener thực hiện các tác vụ follow-up hoặc kích hoạt xử lý tiếp theo.

Guideline cho C&C View

Khi xây dựng C&C View cần tuân thủ các nguyên tắc sau:

- C&C View phải tập trung vào runtime communication thay vì cấu trúc source code.
- Chỉ nên mô tả các runtime component quan trọng và các connector chính giữa chúng.
- Các connector cần thể hiện rõ:
  - synchronous communication.
  - asynchronous communication.
  - event-driven interaction.
- Các request flow chính cần được mô tả rõ ràng từ controller tới service và database.
- Các cơ chế cross-cutting như authentication, messaging hoặc event bus nên được thể hiện rõ trong runtime flow.
- Không nên đưa quá nhiều chi tiết implementation như entity, DTO hoặc class-level dependency vào C&C View.
- Các interaction bất đồng bộ nên được biểu diễn riêng biệt nhằm làm nổi bật kiến trúc event-driven.
- C&C View cần giúp người đọc nhanh chóng hiểu:

- Các runtime component chính của hệ thống.
- Các connector giữa các component.
- Luồng giao tiếp và xử lý request.
- Cơ chế giao tiếp đồng bộ và bất đồng bộ.

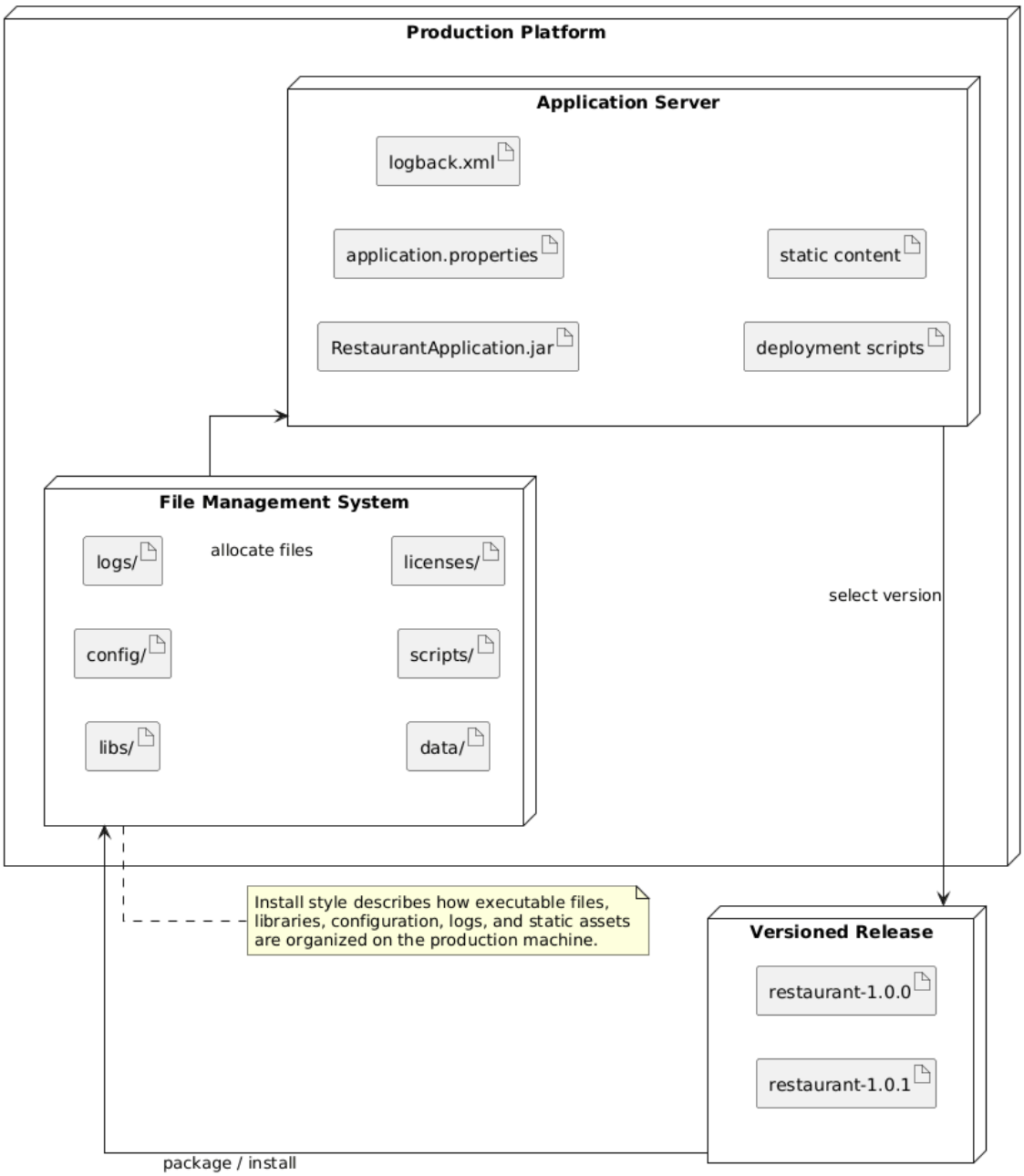
4.3 Allocation View

Allocation View mô tả cách hệ thống phần mềm được ánh xạ tới môi trường triển khai, cấu trúc cài đặt và tổ chức nhân sự phát triển hệ thống. View này giúp thể hiện mối quan hệ giữa kiến trúc phần mềm và môi trường thực tế trong quá trình triển khai, vận hành và phát triển hệ thống.

Trong hệ thống quản lý nhà hàng, Allocation View được mô tả thông qua ba góc nhìn chính:

- Install Style
- Deployment Style
- Work Assignment Style

4.3.1 Install Style



Hình 6: Install Style của hệ thống

Sơ đồ Install Style mô tả cách các thành phần phần mềm, file cấu hình và tài nguyên hệ thống được tổ chức trên môi trường production.

Hệ thống được triển khai trên một *Production Platform* bao gồm hai thành phần chính:

Application Server: Chứa các artifact cần thiết để chạy hệ thống:

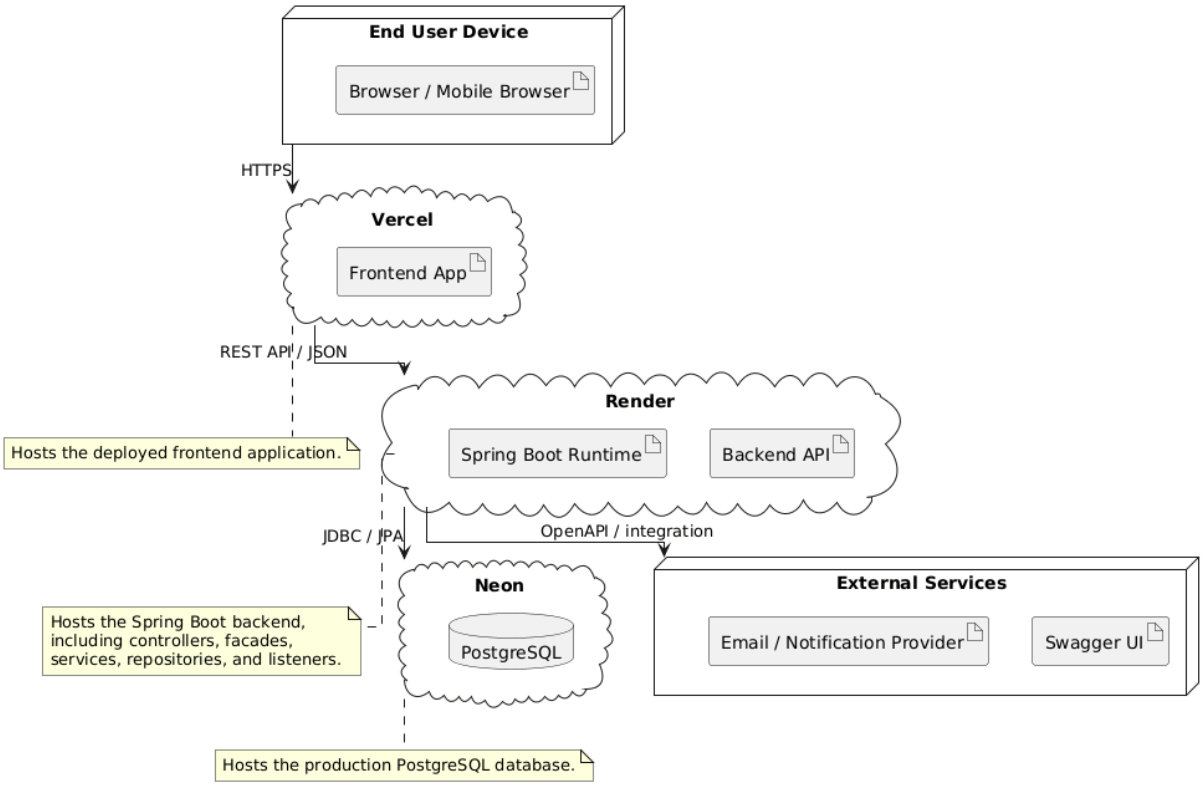
- *RestaurantApplication.jar*: file thực thi chính của ứng dụng Spring Boot.
- *application.properties*: file cấu hình hệ thống.
- *logback.xml*: cấu hình logging.
- *deployment scripts*: script phục vụ triển khai hệ thống.
- *static content*: tài nguyên tĩnh như HTML, CSS, JavaScript và hình ảnh.

File Management System: Quản lý cấu trúc thư mục và các file cài đặt của hệ thống:

- *libs/*: chứa thư viện phụ thuộc.
- *config/*: chứa file cấu hình.
- *logs/*: chứa log runtime.
- *data/*: chứa dữ liệu hệ thống.
- *scripts/*: chứa script triển khai và vận hành.
- *licenses/*: chứa thông tin giấy phép phần mềm.

Ngoài ra, hệ thống còn hỗ trợ quản lý nhiều phiên bản phát hành thông qua các release như: *restaurant-1.0.0* và *restaurant-1.0.1*. Các phiên bản này được package và cài đặt vào môi trường production thông qua File Management System.

4.3.2 Deployment Style



Hình 7: Deployment Style của hệ thống

Sơ đồ Deployment Style mô tả cách hệ thống được triển khai trên các node và nền tảng hạ tầng thực tế.

End User Device: Người dùng truy cập hệ thống thông qua trình duyệt web hoặc mobile browser.

Vercel: Được sử dụng để triển khai frontend application của hệ thống. Frontend chịu trách nhiệm hiển thị giao diện người dùng và gửi request tới backend thông qua REST API.

Render: Được sử dụng để triển khai backend API chạy trên nền tảng Spring Boot Runtime. Backend bao gồm:

- Controllers
- Facades
- Services
- Repositories

- Event listeners

Neon: Được sử dụng để triển khai cơ sở dữ liệu PostgreSQL trên cloud nhằm lưu trữ dữ liệu nghiệp vụ của hệ thống.

External Services: Bao gồm các dịch vụ tích hợp bên ngoài như:

- Swagger UI phục vụ kiểm thử và tài liệu hóa API.
- Email / Notification Provider phục vụ gửi thông báo hệ thống.

Luồng triển khai: Người dùng truy cập frontend trên Vercel thông qua HTTPS, frontend gửi REST API request tới backend trên Render, sau đó backend tương tác với PostgreSQL trên Neon thông qua JDBC/JPA.

4.3.3 Work Assignment Style

Work Assignment Style mô tả việc phân bổ trách nhiệm phát triển các module và nhiệm vụ trong hệ thống cho các nhóm phát triển tương ứng.

Hệ thống được phát triển thông qua nhiều nhóm khác nhau bao gồm:

- Backend Team
- Frontend Team
- QA Team
- DevOps Team

Các nhiệm vụ chính của dự án bao gồm:

- Requirements Analysis
- Architecture Design
- Auth Module
- Menu Module
- Order Module
- Kitchen Module
- Inventory Module

- Frontend Dashboard
- Integration Testing
- Deployment & Monitoring

Mỗi task được phân công:

- Team chịu trách nhiệm chính.
- Team hỗ trợ.
- Thời gian thực hiện.
- Module phụ thuộc.
- Deliverables đầu ra.
- Trạng thái thực hiện.
- Assignee chịu trách nhiệm trực tiếp.

Ngoài ra, Work Assignment Style còn sử dụng Gantt Chart để mô tả timeline phát triển hệ thống từ tuần 1 đến tuần 8. Gantt Chart giúp thể hiện:

- Trình tự thực hiện các task.
- Quan hệ phụ thuộc giữa các module.
- Phân bổ thời gian cho từng nhóm phát triển.
- Tiến độ tổng thể của dự án.

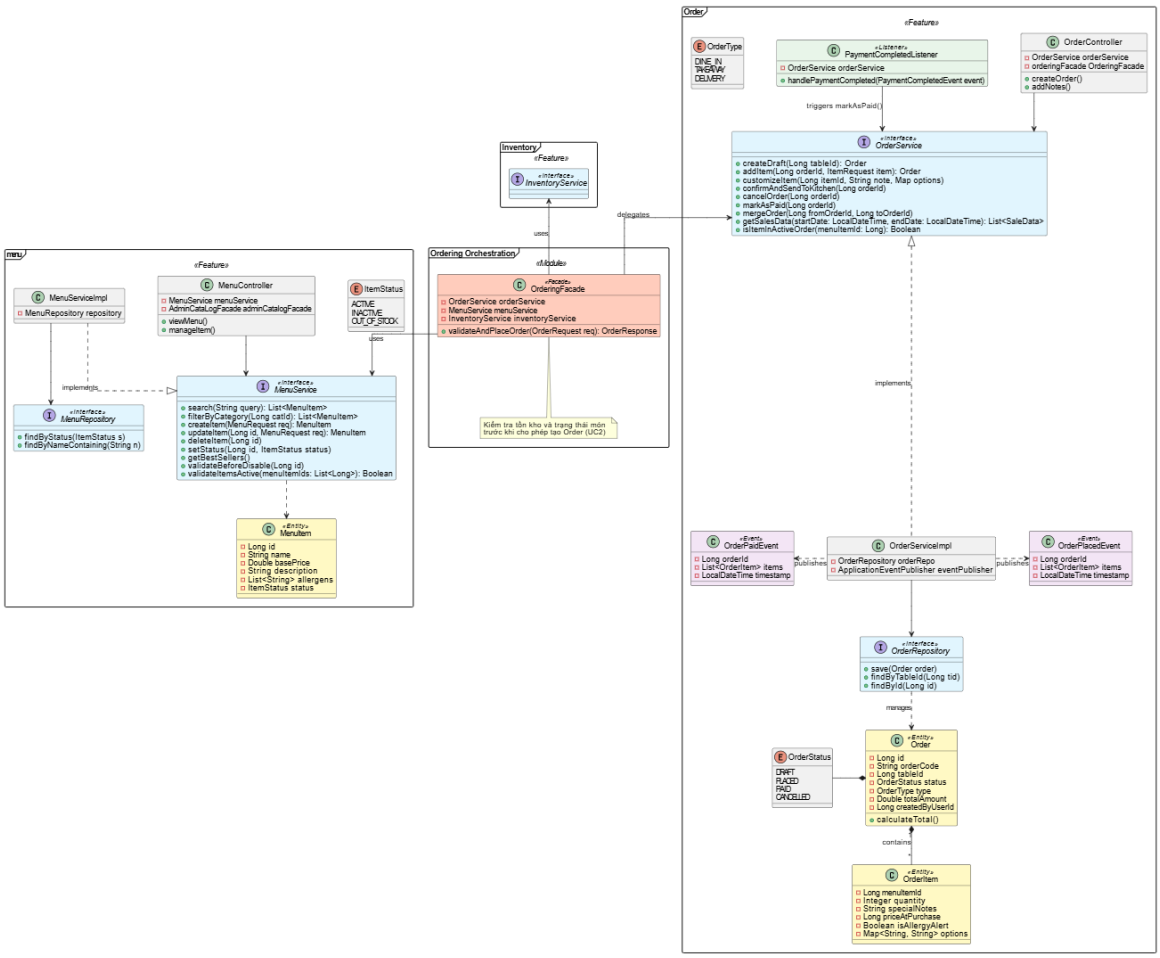
Thông qua Work Assignment Style, nhóm phát triển có thể dễ dàng quản lý tài nguyên, phân chia trách nhiệm và theo dõi tiến độ thực hiện của toàn bộ hệ thống.

5 Thiết kế class diagram

Do thiết kế class diagram quá lớn không thể trình bày trên một trang tài liệu. Vì thế, tài liệu này phân tích thiết kế của class diagram thành những tập hợp các thành phần nhỏ của một số module có liên quan đến nhau cho luồng thực thi nghiệp vụ nào đó.

Thiết kế đầy đủ: Thiết kế Class Diagram

5.1 Nghiệp vụ 1: Nhân viên thực hiện chọn món và tạo order



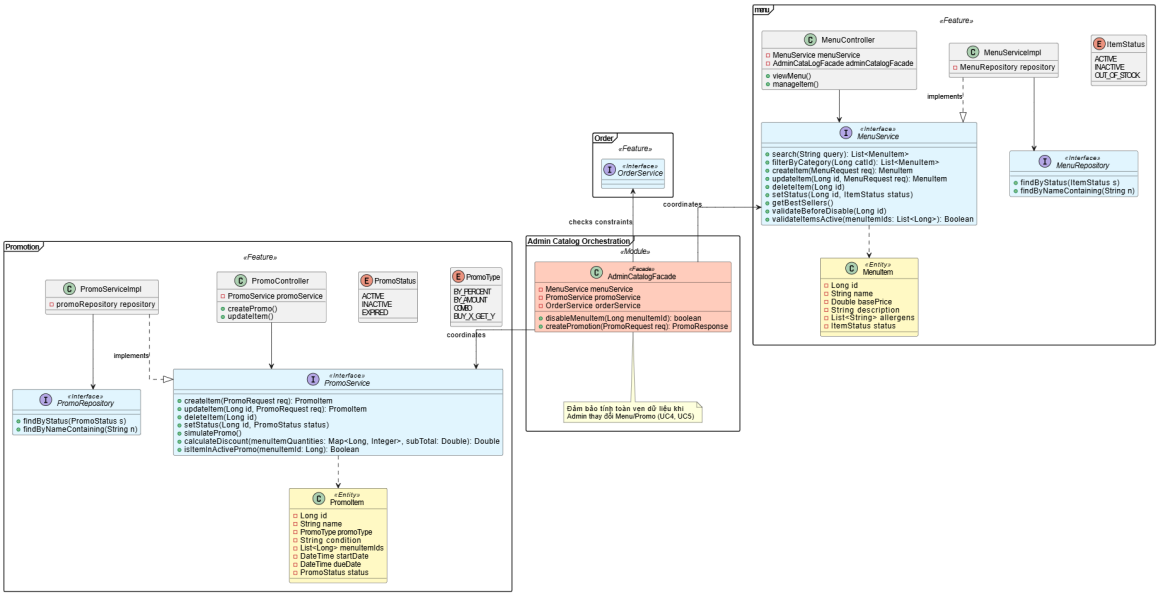
Hình 8: Thiết kế class diagram cho nghiệp vụ tạo order

Khi thực thi các module trong hình sẽ có những tương tác phụ thuộc lẫn nhau như trong hình 1 với các API như sau:

- `createDraft(Long tableId): Order`: trả về một order nháp được khởi tạo. Được gọi trong `createOrder` thuộc class `OrderController`.
- `addItem(Long orderId ItemRequest item): Order`: thực thi khi nhân viên thêm các món ăn vào một order vừa mới tạo.
- `customizeItem(Long itemId, String note, Map options)`: thực thi tạo một OrderItem bao gồm **note** (Lưu ý của khách hàng khi order), **options** (Những yêu cầu bổ sung như topping hay thêm cơm)

Bên cạnh đó, việc thực hiện logic kiểm tra và thực sự tạo một order trong hệ thống sẽ được kiểm tra và quyết định tạo order bởi module **OrderFacade** khác. Thiết kế này đảm bảo nguyên lý Single Responsibility trong SOLID, khi module Order không cần biết việc kiểm tra menuItem hợp lệ diễn ra thế nào. Ngoài ra, việc đặt code được hiện thực ra một module khác như vậy giúp dễ bảo trì logic kiểm tra trạng thái của một thực thể thuộc module khác.

5.2 Nghiệp vụ 2: Quản trị viên thiết lập Catalog (Menu và Khuyến mãi)



Hình 9: Thiết kế class diagram cho nghiệp vụ quản lý danh mục và khuyến mãi

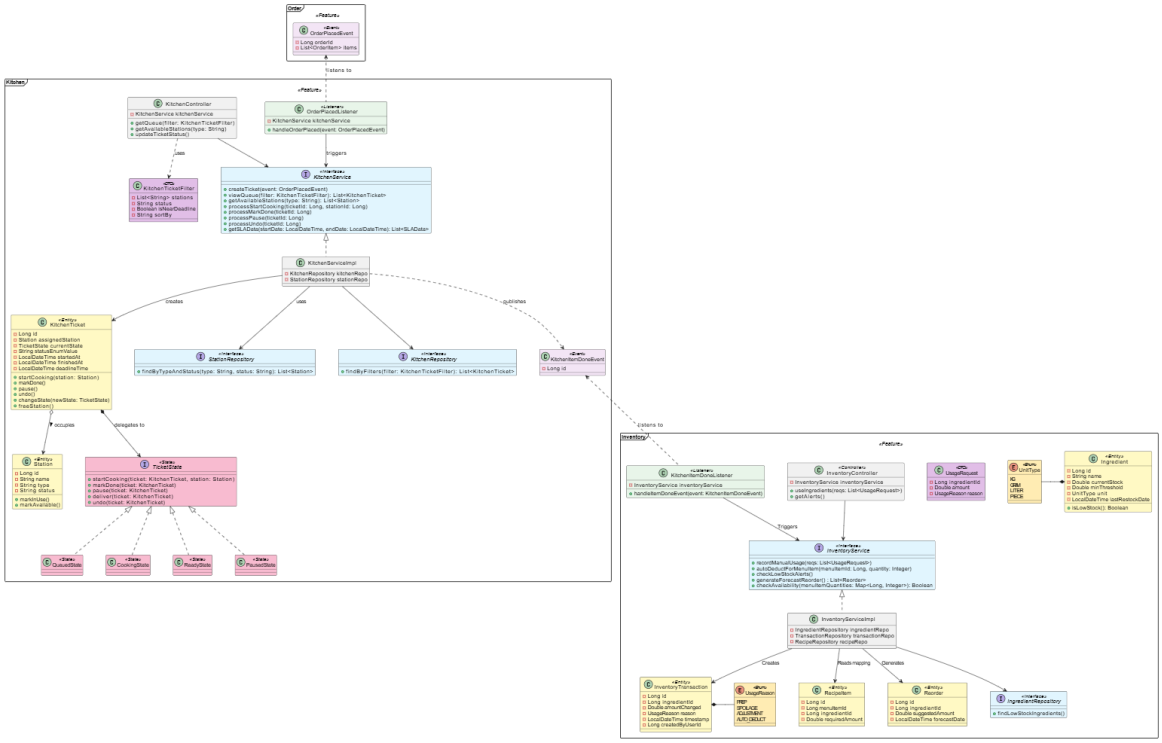
Nghiệp vụ quản lý thực đơn và các chương trình khuyến mãi đòi hỏi tính toàn vẹn dữ liệu cao giữa các phân hệ. Các API chính tham gia vào tiến trình này bao gồm:

- `createItem(MenuRequest req): MenuItem` và `createItem(PromoRequest req): PromoItem`: tạo mới các thực thể món ăn hoặc chương trình khuyến mãi thông qua các Service tương ứng.
- `setStatus(Long id, ItemStatus status)`: thực hiện cập nhật trạng thái hoạt động của thực đơn (Active, Inactive, Out of Stock).
- `disableMenuItem(Long menuItemId): boolean`: API được bọc bởi **AdminCatalogFacade** để thực hiện vô hiệu hóa một món ăn một cách an toàn.

Để tránh việc dữ liệu bị lỗi (ví dụ: xóa một món ăn đang nằm trong một chương trình khuyến mãi đang chạy hoặc một order chưa thanh toán), hệ thống áp dụng **Facade Pattern** thông qua `AdminCatalogFacade`. Module này đóng vai trò điều phối, sẽ thực hiện gọi chéo sang `OrderService` và `PromoService` để xác thực các ràng buộc nghiệp

vụ trước khi ra lệnh cho `MenuService` cập nhật cơ sở dữ liệu.

5.3 Nghiệp vụ 3: Xử lý nhà bếp và Tự động trừ tồn kho



Hình 10: Thiết kế class diagram cho nghiệp vụ chế biến và quản lý nguyên liệu

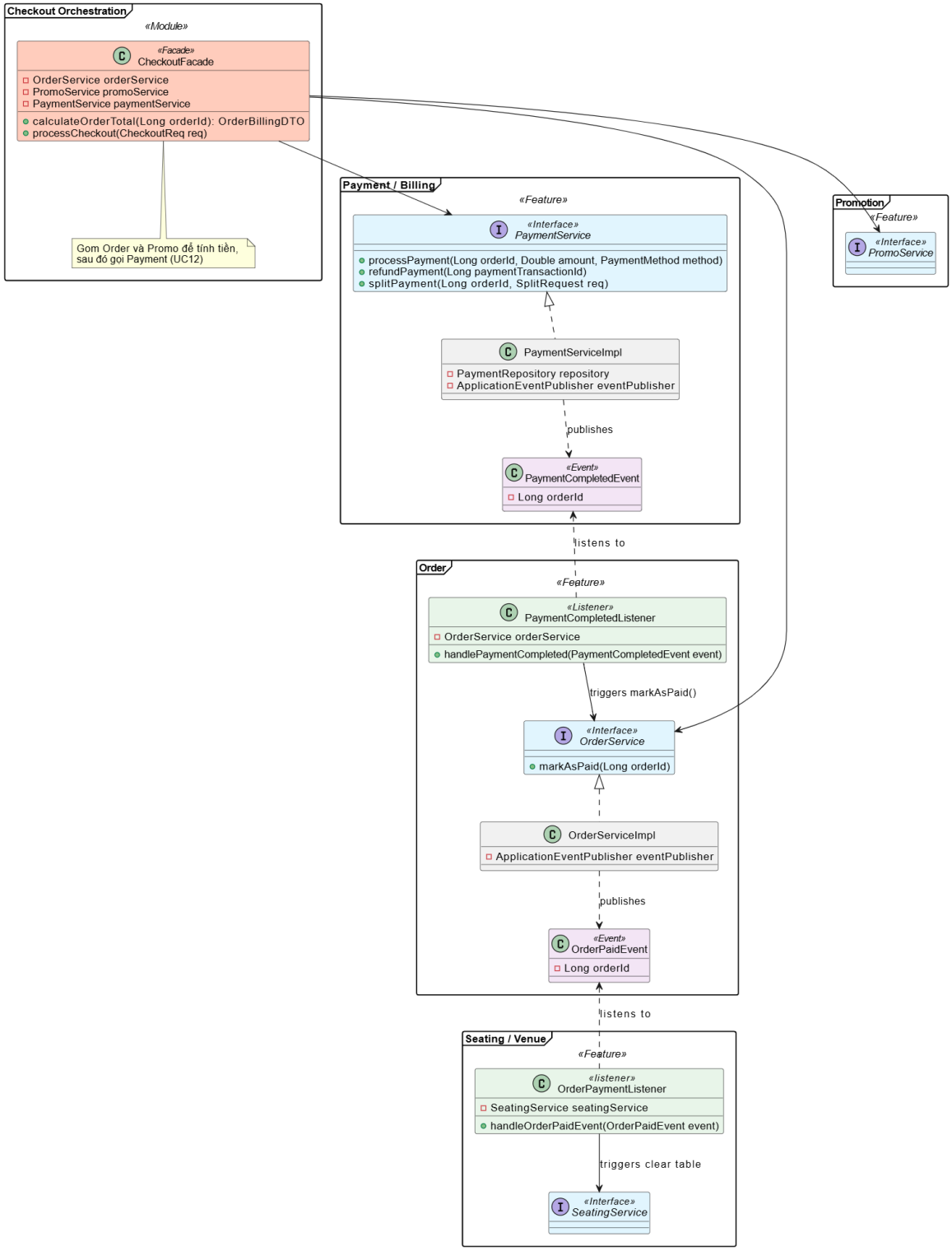
Đây là luồng nghiệp vụ hoạt động chủ yếu ngầm ở phía sau (Back of House), phụ thuộc mạnh vào cơ chế giao tiếp bất đồng bộ thông qua Event. Các API và hành vi chính bao gồm:

- `handleOrderPlaced(event: OrderPlacedEvent)`: Listener phía Kitchen nhận tín hiệu từ luồng Order, sau đó gọi `createTicket` để tạo danh sách công việc cho đầu bếp.
- `changeState(newState: TicketState)`: quản lý vòng đời của một phiếu nấu ăn thông qua **State Pattern** (từ `QueuedState` sang `CookingState` và cuối cùng là `ReadyState`).
- `processMarkDone(ticketId: Long)`: Đầu bếp xác nhận hoàn thành món ăn. Hàm này sẽ phát ra sự kiện `KitchenItemDoneEvent`.
- `autoDeductForMenuItem(menuItemId: Long, quantity: Integer)`: Listener phía Inventory nhận sự kiện hoàn thành món, tra cứu `RecipeItem` để

trừ nguyên liệu thô tự động.

Sự liên kết lỏng lẻo (loose coupling) ở đây mang lại lợi ích: các thao tác của nhà bếp hoàn toàn độc lập với kho. Sự tách biệt module và sử dụng **Observer Pattern** để giao tiếp giúp code dễ maintain theo từng module cũng như giảm sự phụ thuộc trong luồng thực thi giữa cả 2 module.

5.4 Nghiệp vụ 4: Thanh toán hóa đơn và Quản lý trạng thái bàn



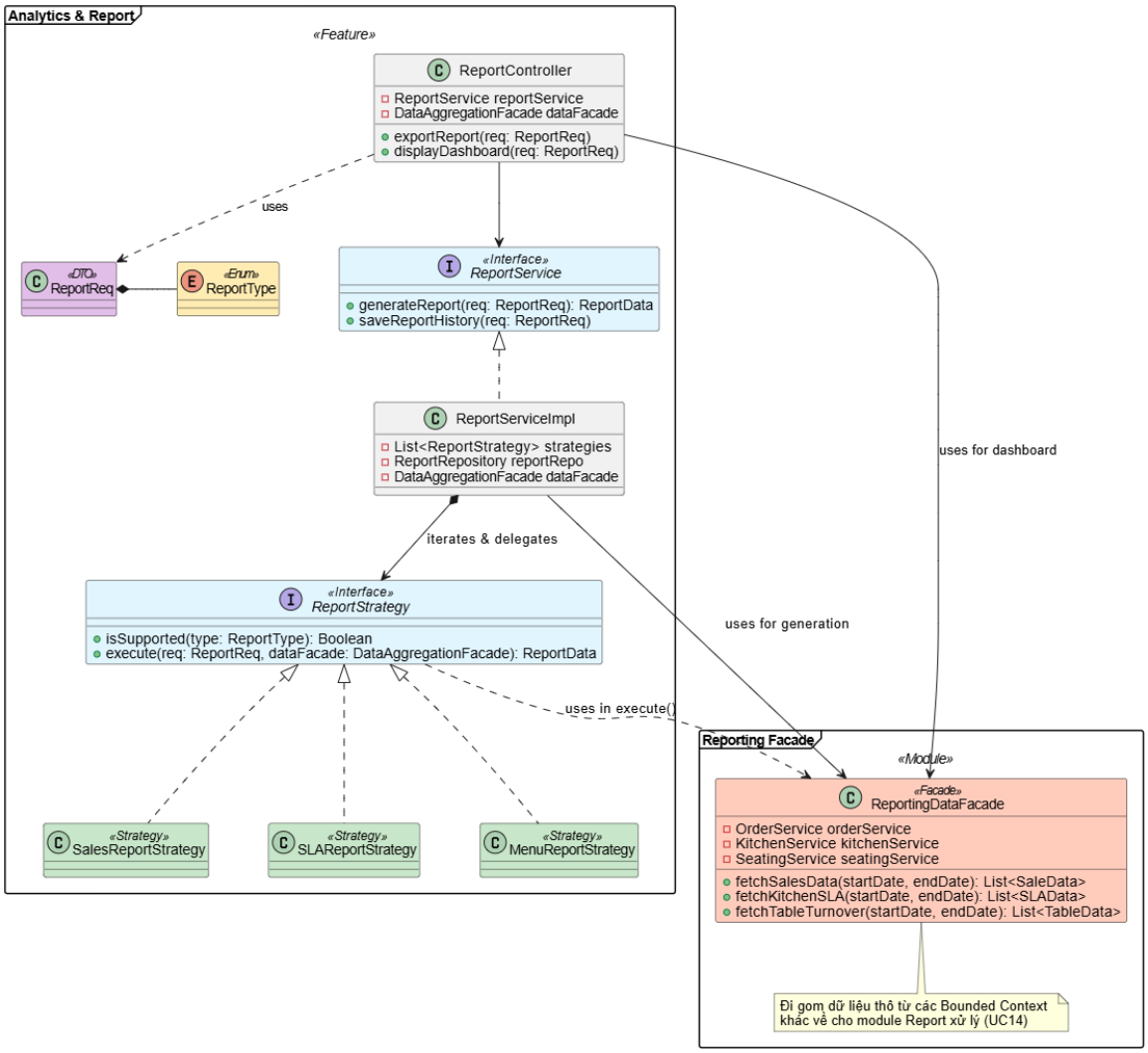
Hình 11: Thiết kế class diagram cho nghiệp vụ thanh toán và giải phóng bàn

Giai đoạn cuối cùng của chu trình phục vụ khách hàng bao gồm việc tính toán chi phí, xử lý thanh toán và cập nhật sơ đồ nhà hàng.

- `calculateOrderTotal(Long orderId)`: được gọi qua `CheckoutFacade` nhằm gom dữ liệu từ `OrderService` và tính toán mức giảm giá thông qua `PromoService`.
- `processPayment(Long orderId, Double amount, PaymentMethod method)`: thực thi giao dịch thanh toán và phát ra `PaymentCompletedEvent` khi thành công.
- `markAsPaid(Long orderId)`: `OrderService` lắng nghe giao dịch thành công để cập nhật trạng thái hóa đơn và tiếp tục kích hoạt `OrderPaidEvent`.
- `markAsDirty()`: API thuộc `Table` entity, được trigger bởi `OrderPaymentListener` thuộc module Seating để tự động báo cho nhân viên dọn bàn.

Việc áp dụng `CheckoutFacade` giúp hệ thống giấu đi sự phức tạp của việc tính toán giá trị hóa đơn (bao gồm thuế, tip, khuyến mãi chồng chéo). Đồng thời, phản ứng dây chuyền qua Event (Payment → Order → Seating) đảm bảo sơ đồ nhà hàng (Floor plan) giúp tạo ra giao tiếp tự động giữa các module.

5.5 Nghiệp vụ 5: Phân tích dữ liệu và Xuất báo cáo



Hình 12: Thiết kế class diagram cho nghiệp vụ phân tích và tổng hợp báo cáo

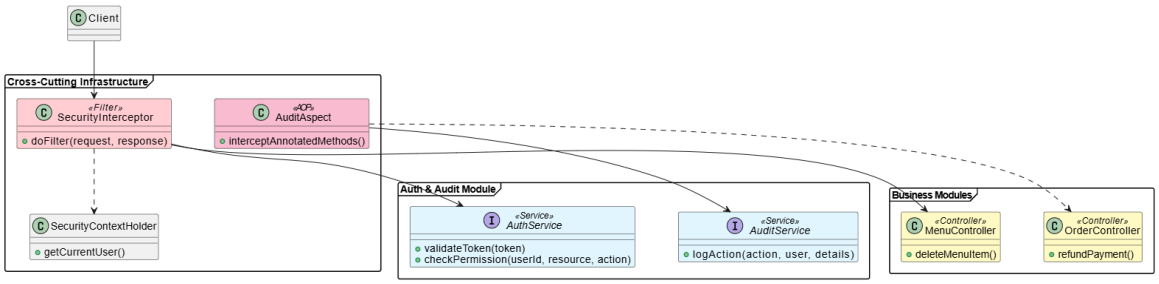
Module Analytics hoạt động độc lập và không can thiệp vào tiến trình giao dịch của các phân hệ khác (Non-blocking).

- `generateReport(req: ReportReq): ReportData`: điểm vào duy nhất cho mọi yêu cầu báo cáo.
- `execute(req, dataFacade)`: phương thức thuộc giao diện `ReportStrategy`. Tùy thuộc vào loại báo cáo, hệ thống sẽ gọi `SalesReportStrategy`, `SLAReportStrategy` hoặc `MenuReportStrategy`.
- `fetchSalesData`, `fetchKitchenSLA`: Các API gom dữ liệu thô từ các module

nghiệp vụ khác thông qua `ReportingDataFacade`.

Thiết kế kết hợp giữa **Strategy Pattern** và **Facade Pattern** giúp hệ thống Report dễ dàng mở rộng. Khi nhà hàng cần thêm một loại báo cáo mới (ví dụ: Báo cáo nhân sự), ta chỉ cần tạo thêm một Strategy class mới mà không làm ảnh hưởng đến mã nguồn cũ (tuân thủ nguyên lý Open/Closed trong SOLID).

5.6 Nghiệp vụ 6: Kiểm soát truy cập và Lưu vết hệ thống (Cross-Cutting)



Hình 13: Thiết kế class diagram cho hạ tầng phân quyền và Audit

Để đảm bảo tính bảo mật và truy vết (đặc biệt trong các thao tác nhạy cảm như hoàn tiền, xóa món), hệ thống áp dụng kiến trúc Cross-Cutting Concerns.

- `doFilter(request, response)`: Thuộc `SecurityInterceptor`, chặn mọi luồng API đi vào hệ thống để xác thực Token và kiểm tra vai trò người dùng qua `AuthService`.
- `interceptAnnotatedMethods()`: Thuộc `AuditAspect`, tự động bao bọc các Controller nhạy cảm.
- `logAction(action, user, details)`: Ghi nhận lịch sử thao tác vào cơ sở dữ liệu.

Việc sử dụng **AOP (Aspect-Oriented Programming)** thông qua `AuditAspect` là một thiết kế tối ưu. Nó giúp bóc tách hoàn toàn logic kiểm tra quyền và ghi log ra khỏi logic nghiệp vụ chính (Business Logic) bên trong các Service. Nhờ đó, mã nguồn của `OrderController` hay `MenuController` giữ được sự gọn gàng và dễ đọc.